

Point Processing & Filtering

CS 4243 Computer Vision & Pattern Recognition

Angela Yao

Recap & Outline

Last Week

- What is computer vision
- Applications in computer vision
- Why computer vision is difficult
- Historical development of the field
- RGB & HSV colour spaces
- Digital image capture, representations and compression

Today's Lecture

- Point processing operations:
- Histogram stretching & equalization
- Cross-correlation & convolution
- Denoising, sharpening, template matching
- Non-linear filtering

Point Processing

Brightness, contrast, normalization, gamma, logarithmic & exponential mapping

Image histograms, histogram stretching & equalization

Notation

$\mathbf{P}$: 2D array of pixel data

p_{ij} : element at i^{th} of I rows, j^{th} of J columns

$\mathbf{w}_{ij}$: window of data centered at i, j
(0,0) j J



Transformations

Point processing

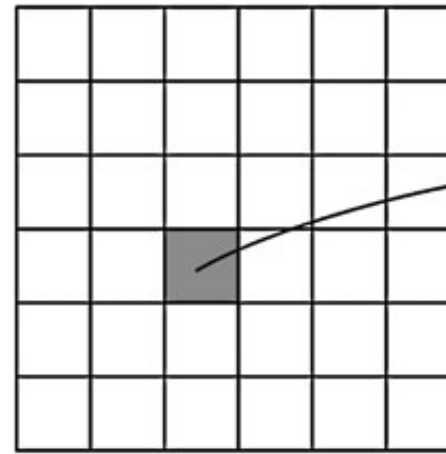
$$x_{ij} = f(p_{ij})$$

corresponding pixel
from input image

Filtering

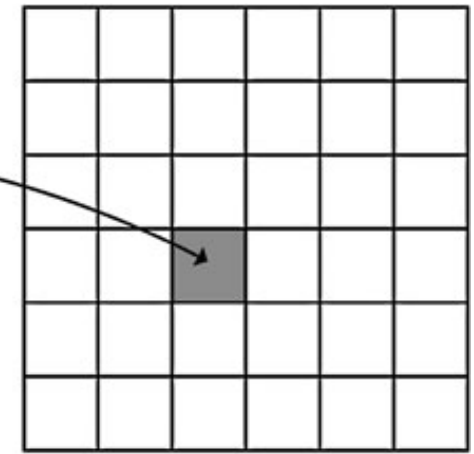
$$x_{ij} = f(\mathbf{w}_{ij})$$

local window of
data from input

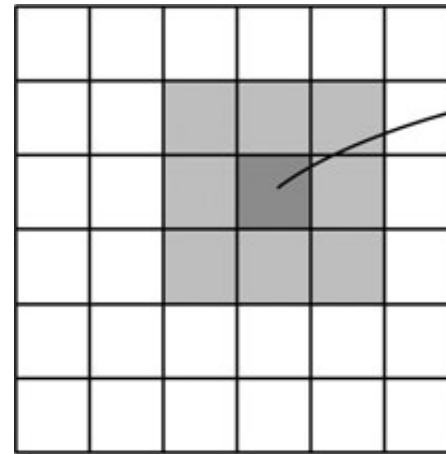


P: input image with elements p_{ij}

operation

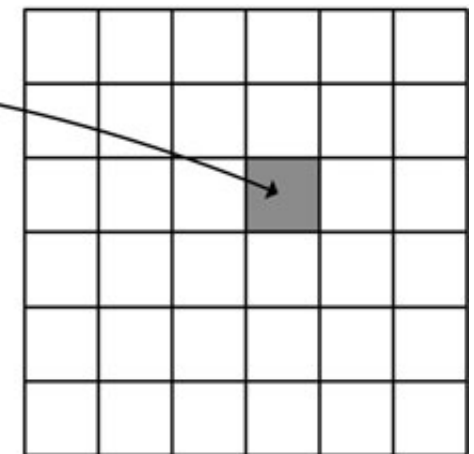


X: output image with elements x_{ij}



P: input image with elements p_{ij}

operation

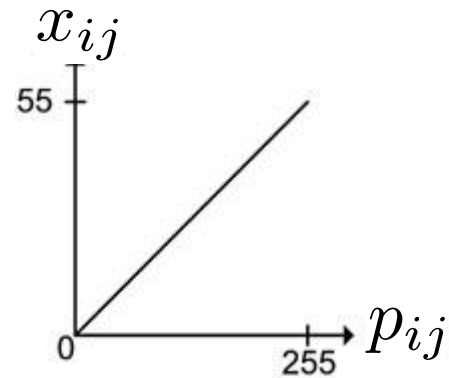


X: output image with elements x_{ij}



Adjusting Brightness

$$x_{ij} = p_{ij} + b$$

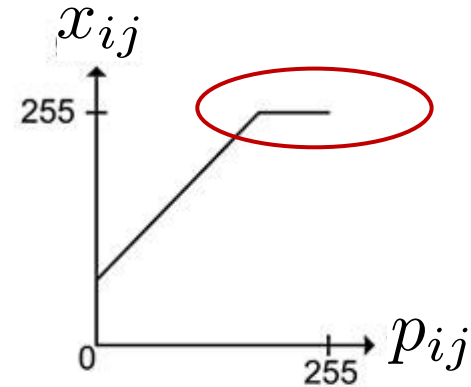


original image



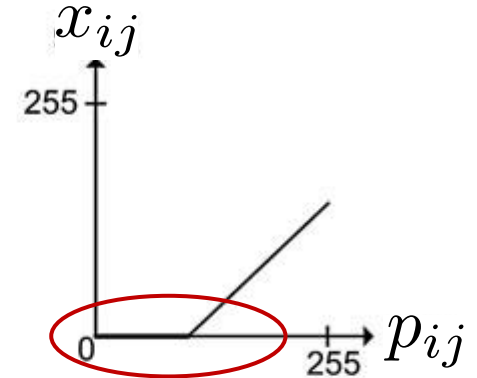
$g(x,y), b = 0$

no change



$g(x,y), b = 75$

increase brightness, $b > 0$



$g(x,y), b = -100$

decrease brightness, $b < 0$

Intensity Scaling

$$x_{ij} = a \cdot p_{ij}$$



Decreased Contrast

$$a < 1$$



original image

$$a = 1$$



Increased Contrast

$$a > 1$$



Image Standardization

$$\text{mean } \mu = \frac{\sum_{i=1}^I \sum_{j=1}^J p_{ij}}{IJ}$$

$$\text{variance } \sigma^2 = \frac{\sum_{i=1}^I \sum_{j=1}^J (p_{ij} - \mu)^2}{IJ}$$

resulting image pixels are
zero-mean, unit variance

$$x_{ij} = \frac{p_{ij} - \mu}{\sigma}$$



before

← This transformation helps
in numerical operations, but
does not help visual fidelity →



after



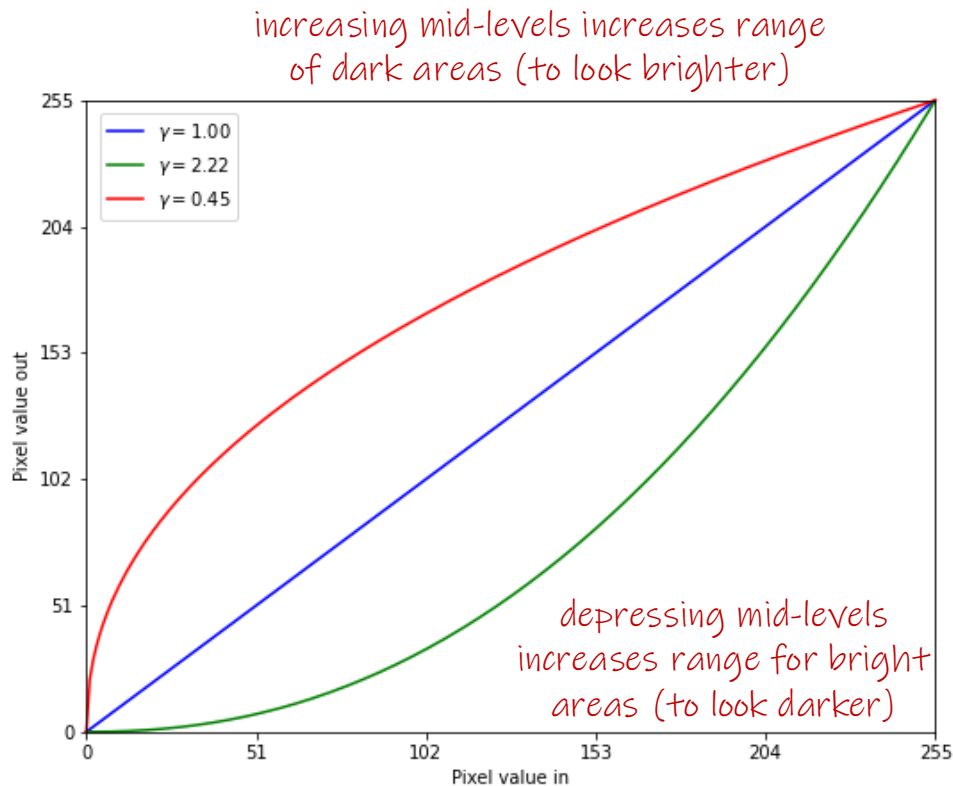
Gamma mapping

Gamma Mapping

Mappings can also be non-linear, e.g.

$$x_{ij} = 255 \cdot \left(\frac{p_{ij}}{255} \right)^\gamma, \quad \gamma > 0$$

p_{ij} is $[0, 255]$,
the scaling
term is $[0, 1]$



No gamma correction



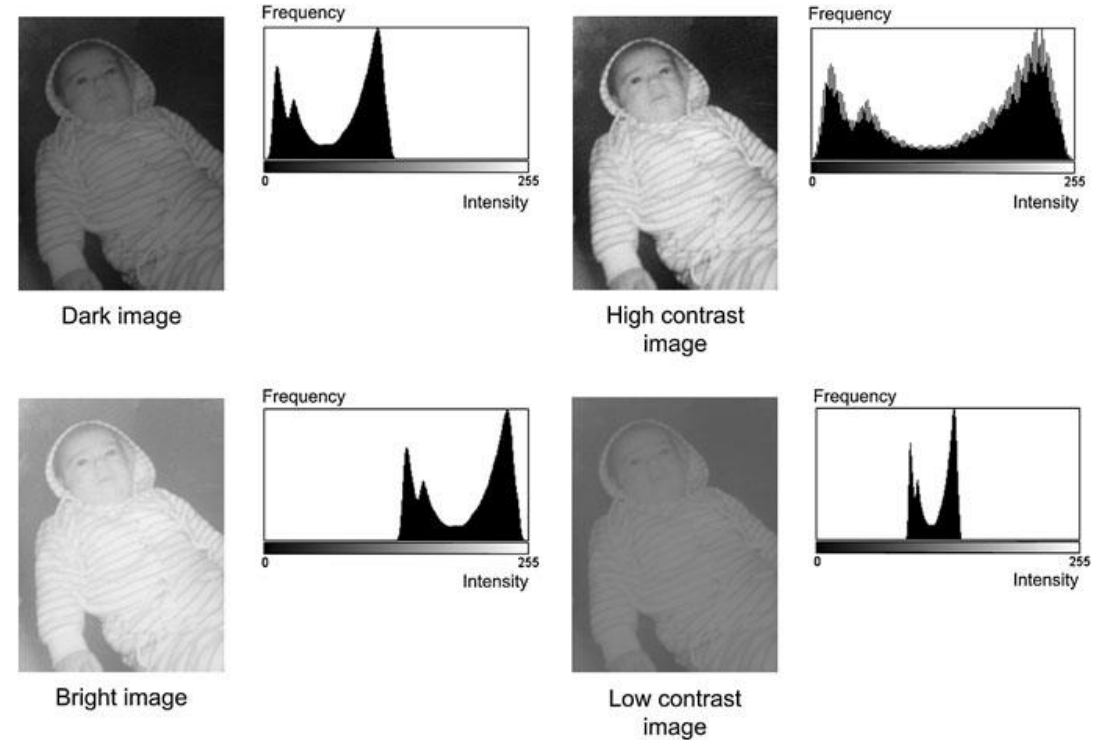
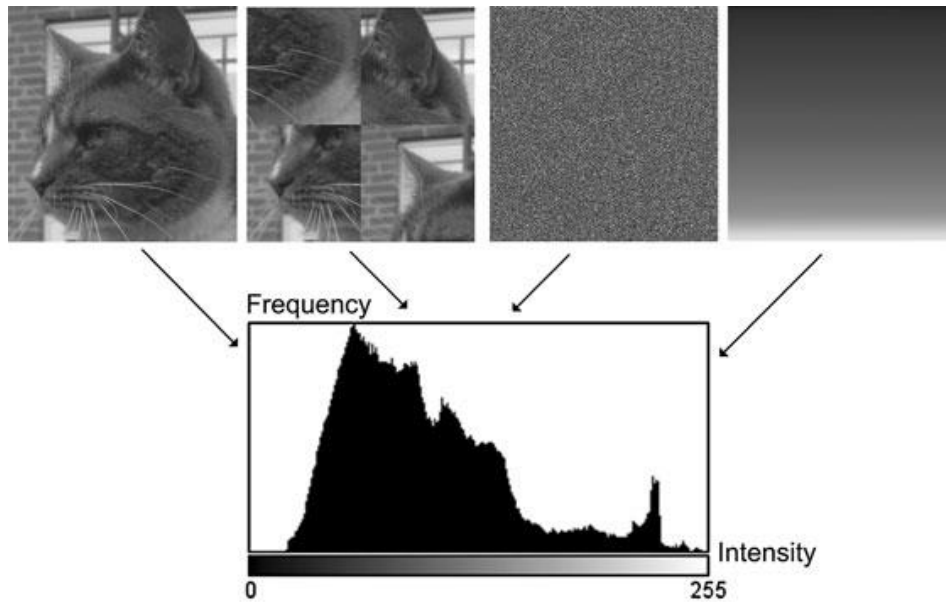
Gamma value: 0.45



Gamma value: 2.22

Image Histogram

- Information is global (pixel locations don't matter)



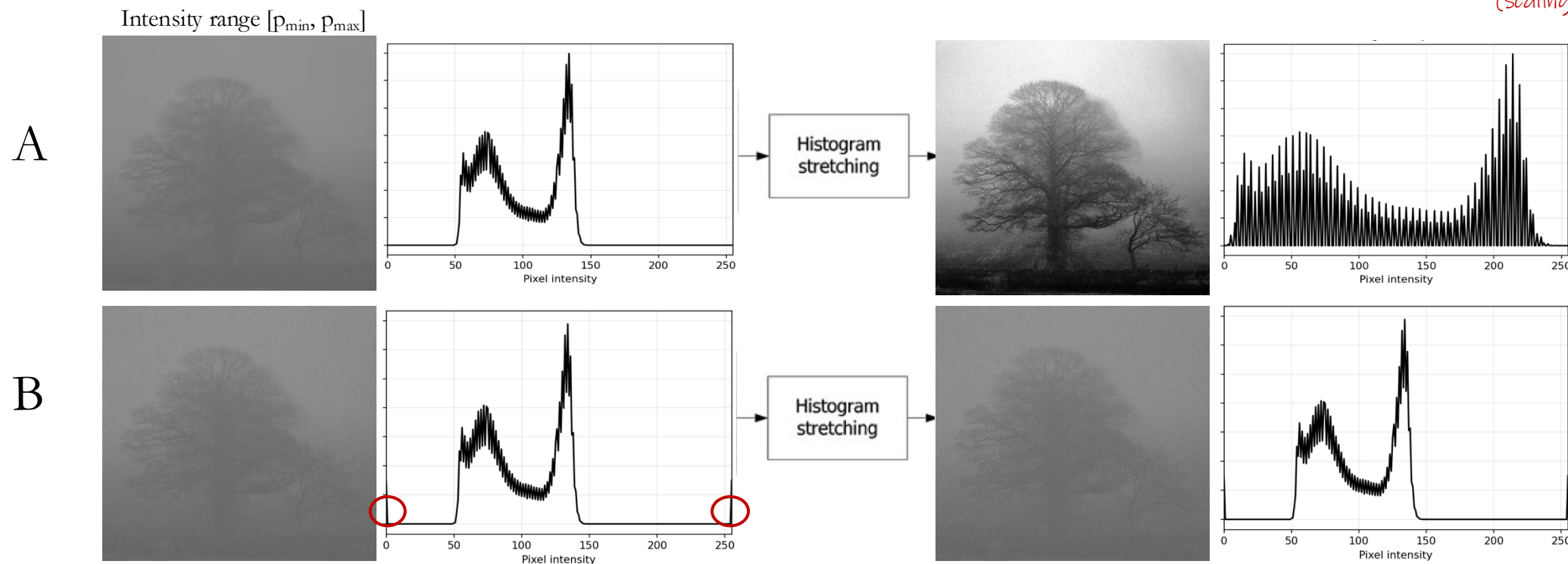
Histogram Stretching

L is the # of intensity levels, e.g. 256

$$x = \underbrace{(p - p_{min})}_{\text{offset term that shifts } p_{min} \text{ to 0 (adjusting brightness)}} \underbrace{\frac{L - 1}{p_{max} - p_{min}}}_{\text{scales existing range (} p_{max} - p_{min} \text{) to the full range of values (scaling intensity)}}$$

How can we linearly map p_{min} to 0 and p_{max} to 255?

Linearly adjusting the brightness and scaling the intensity.



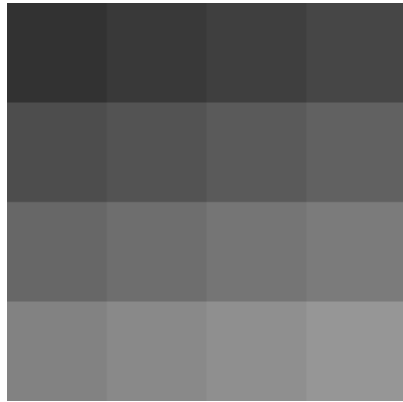
What is the difference between A and B?

The linear mapping doesn't care about the internal distribution of points between p_{min} and p_{max}

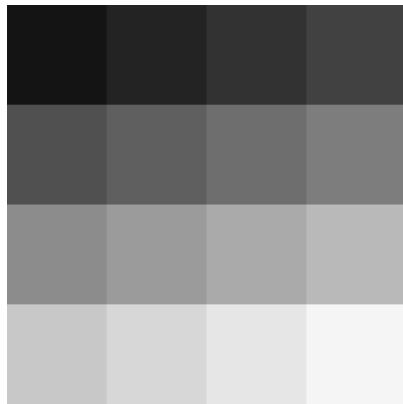
Equalization as percentile mapping

Histogram equalization maps an input intensity according to the cumulative distribution, i.e. the proportion of pixels at or below that intensity.

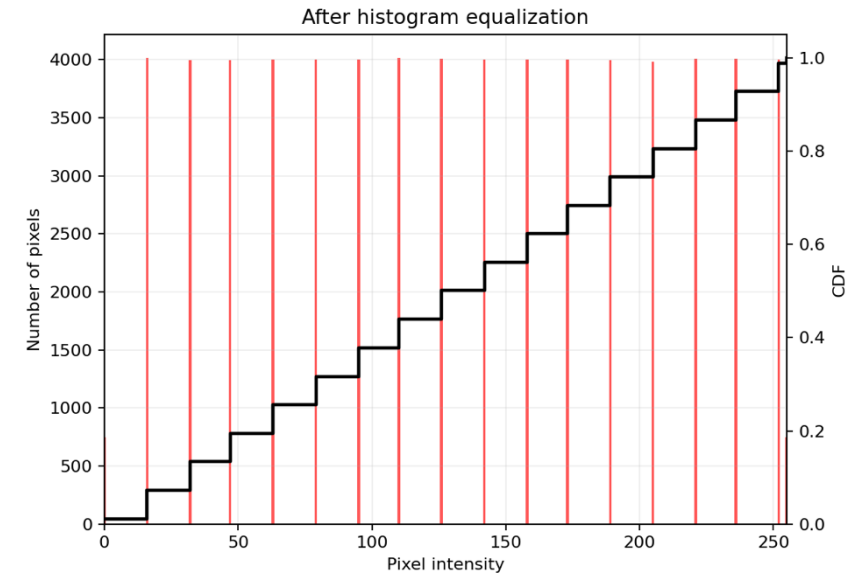
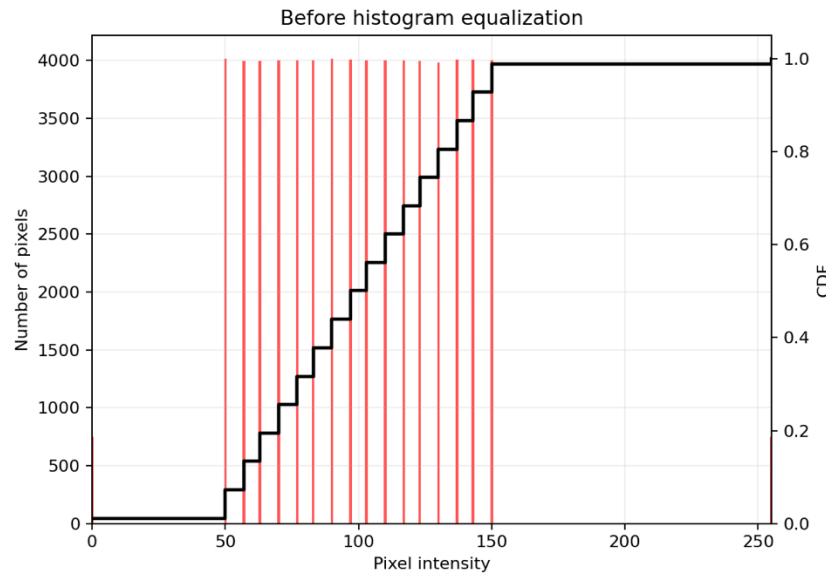
$$Intensity_{output} = k * CDF(Intensity_{input})$$



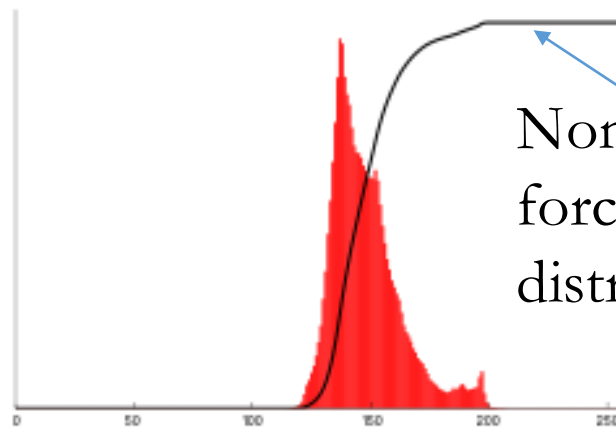
Before



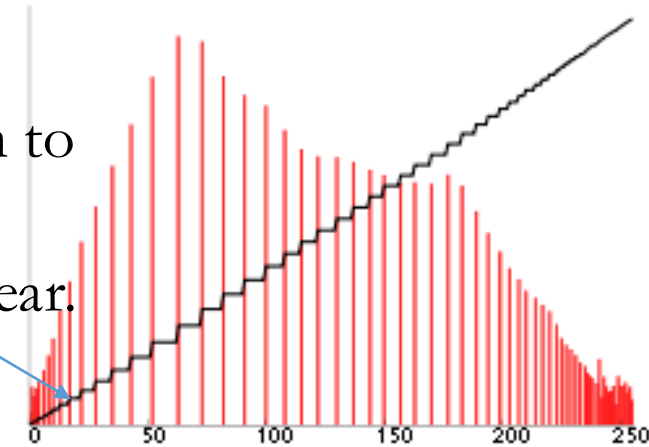
After



Histogram Equalization

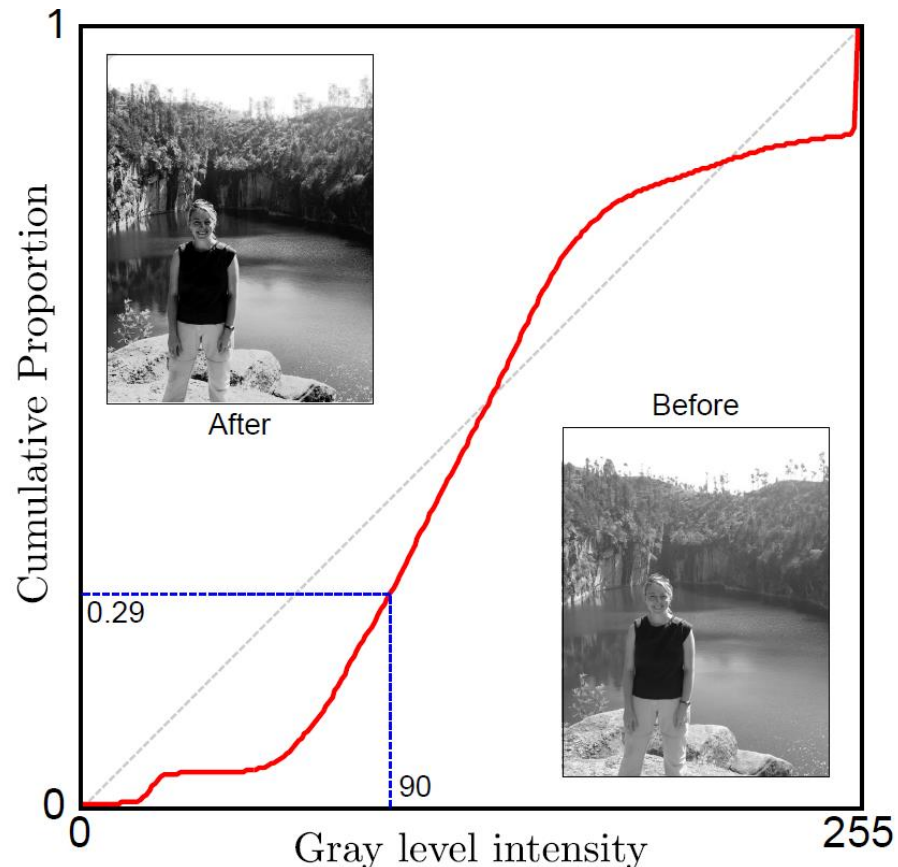


Non-linear operation to
force the cumulative
distribution to be linear.



Histogram Equalization

Forces the cumulative distribution to be linear.



histogram

$$h_k = \sum_{i=1}^I \sum_{j=1}^J \delta[p_{ij} - k],$$

Cumulative sum
of histogram
("CDF")

$$c_k = \frac{\sum_{l=1}^k h_l}{IJ}.$$

K is usually the
maximum intensity,
e.g. 255

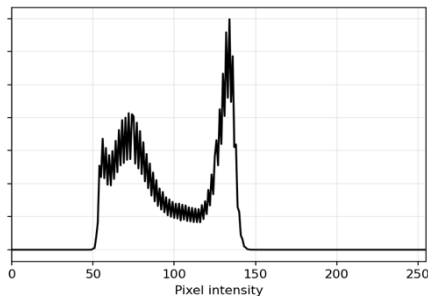
$$x_{ij} = K c_{p_{ij}}.$$

Look up for each pixel intensity, the
corresponding cumulative proportion value
and remap e.g. 90 maps to $255 \times 0.29 = 74$

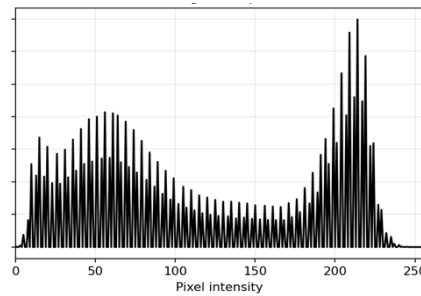


Equalization vs. Stretching

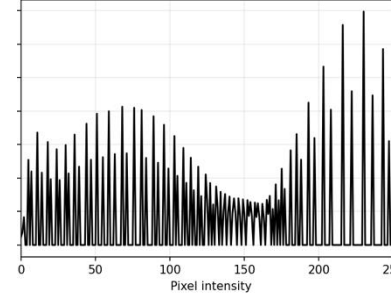
Original



Stretched



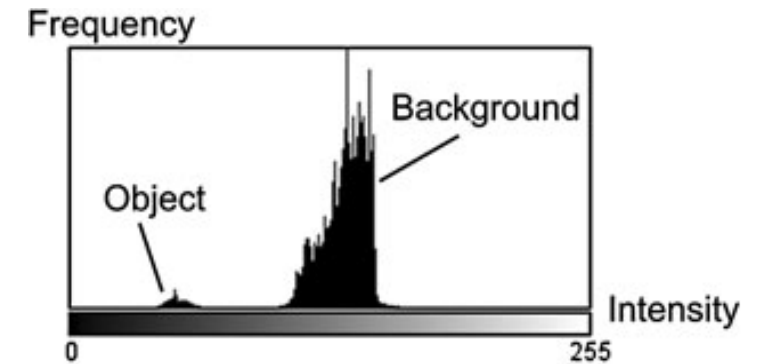
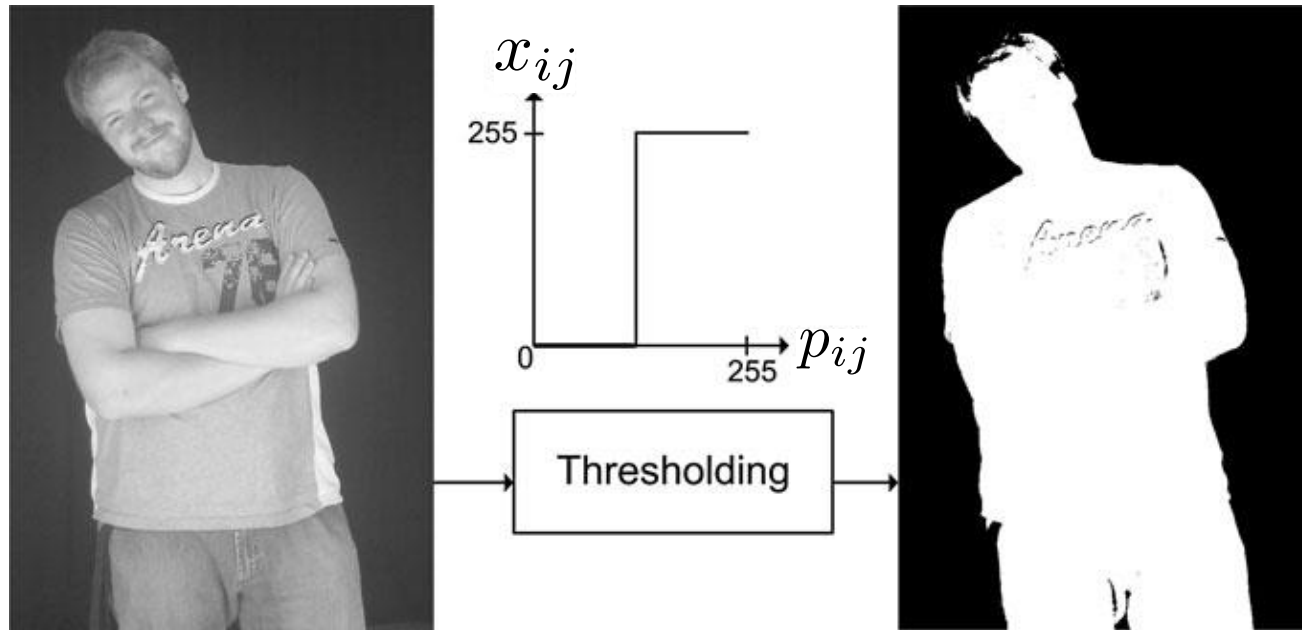
Equalized



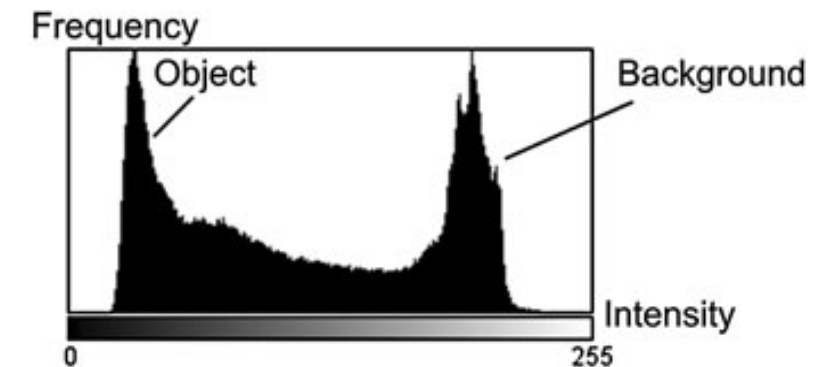
- Stretching applies a **linear transformation** of intensities
- Equalization uses a **nonlinear transformation** based on the cumulative distribution
- Stretching improves contrast while largely preserving relative brightness relationships; equalization can produce a **more dramatic contrast change**

Note: Independently equalizing R, G and B channels may distort colour, so equalization is usually applied to a luminance/value-like component.

What are Histograms Good For?

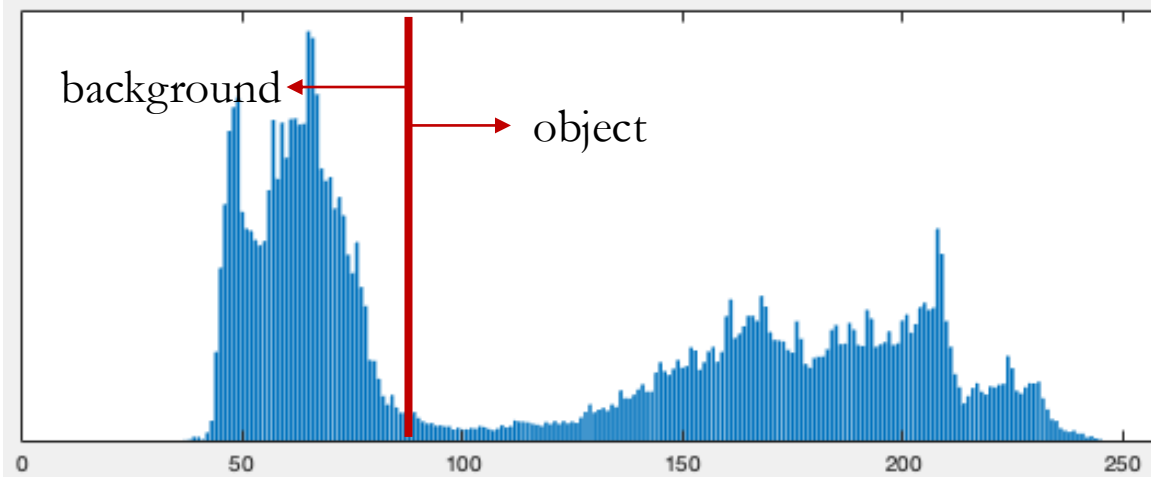


Ideal histogram



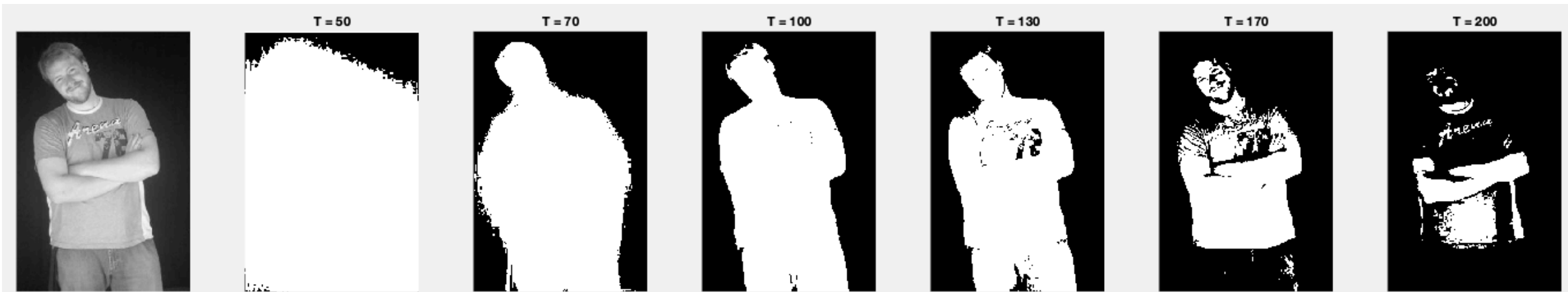
Problematic histogram

Histogram Thresholding for FG/BG Separation

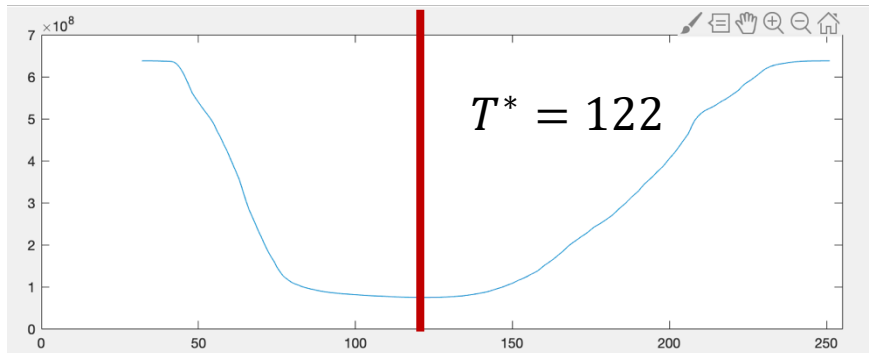


The threshold which separates the object from the background depends on the image. Can we choose the right threshold automatically?

If the image intensity histogram is bi-modal (two “humps”), then the threshold should sit somewhere in between the two modes.



Otsu's Method – Automated Thresholding



T=122



$$T^* = \underset{T}{\operatorname{argmin}} w_1(T) \cdot \sigma_1^2(T) + w_2(T) \cdot \sigma_2^2(T)$$

- optimal threshold T^* minimizes the sum of weighted variances of object and background.
- rationale:
 - correct threshold produces two narrow modes
 - incorrect threshold produces (at least one) wide mode
 - width of the mode is measured by the variance σ^2
 - weighting by number of pixels in each mode
- $\sigma_1^2(T)$, $\sigma_2^2(T)$ - variance of pixels less than or equal to and greater than threshold respectively
- $w_1(T)$, $w_2(T)$ - number of pixels less than or equal to and greater than threshold
- works best when foreground and background form reasonably distinct intensity groups



In-Lecture Question:



Which of the following is true about histogram stretching and histogram equalization?

- A. Stretching is non-linear; equalization is linear.
- B. Stretching uses the extremes, while equalization uses the distribution.
- C. Stretching always preserves colour.
- D. Equalization makes the histogram perfectly uniform.

Correlation Filters

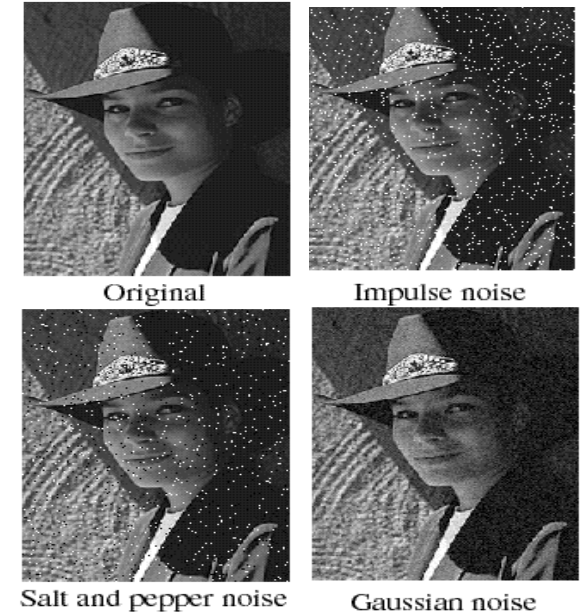
Types of image noise; cross-correlation operation

Box & Gaussian smoothing; sharpening & template matching

Motivation: Noise

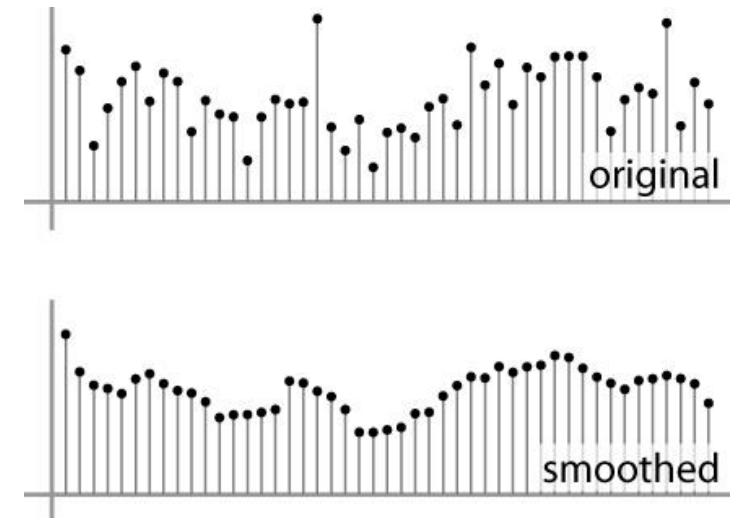
What causes noise?

- **Impulse & salt and pepper noise:** random occurrences of white (and black) pixels, usually due to defective sensor elements
- **Gaussian noise:** variations in intensity drawn from a Gaussian distribution, due to inherent noise of the sensor



How can we deal with it?

- Smoothing by averaging with neighboring pixels, either uniformly (every neighbour the same) or custom weights (e.g. further neighbours count less)
- Smoothing can increase visual quality but leads to loss in detail



Correlation filtering

Generalizes weighted sum, with different weights depending on neighboring pixel's relative position:

$$x_{ij} = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{f_{uv}}_{\text{Weight depends on neighbour's relative position wrt } p_{ij}} \cdot p_{i+u, j+v}$$

For simplicity, consider a square of $(2k+1) \times (2k+1)$; allow more unusual neighbourhood via 0 weights.

This is called cross-correlation, denoted $\mathbf{X} = \mathbf{P} \otimes \mathbf{F}$

Cross-correlation is a linear operation.

Filter Kernel
(stores the weights).

Averaging filter (Box filter)

Which values belong in the kernel F for the moving average example?

$$\mathbf{P} \otimes \mathbf{F} = \mathbf{X}$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

“box filter”

	0	10	20	30	30				

Gaussian filter

What if we want nearest neighboring pixels to have the most influence on the output?

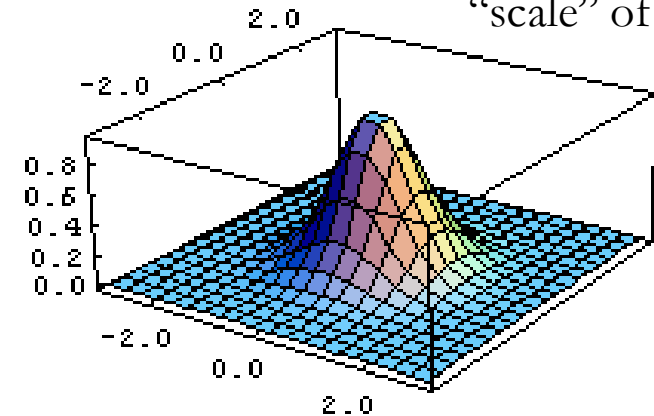
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

1	2	1
2	4	2
1	2	1

Note the weights are normalized to sum to 1.

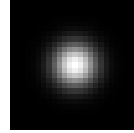
$$f_{uv} = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

“scale” of Gaussian

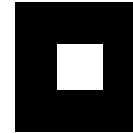


Both the box filter and the Gaussian filter has a smoothing effect.

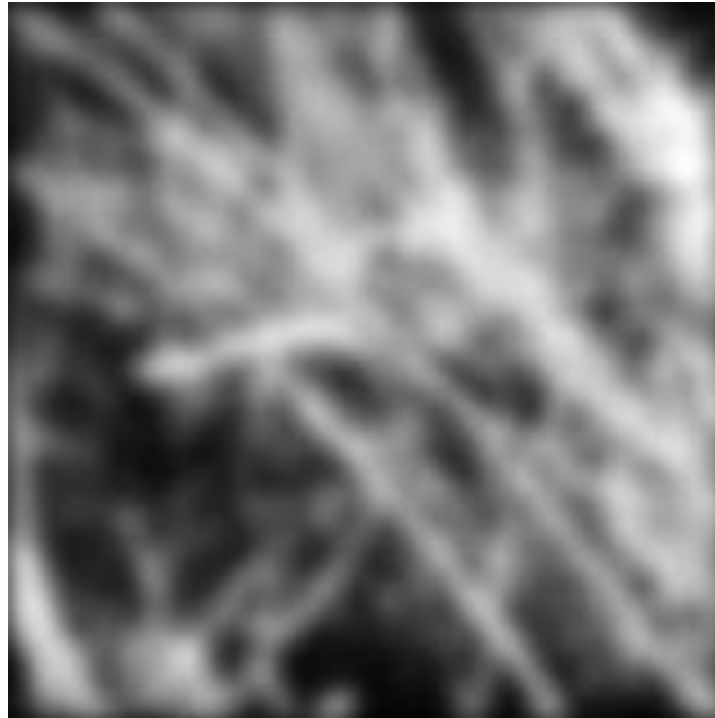
Smoothing with Gaussian vs. Box



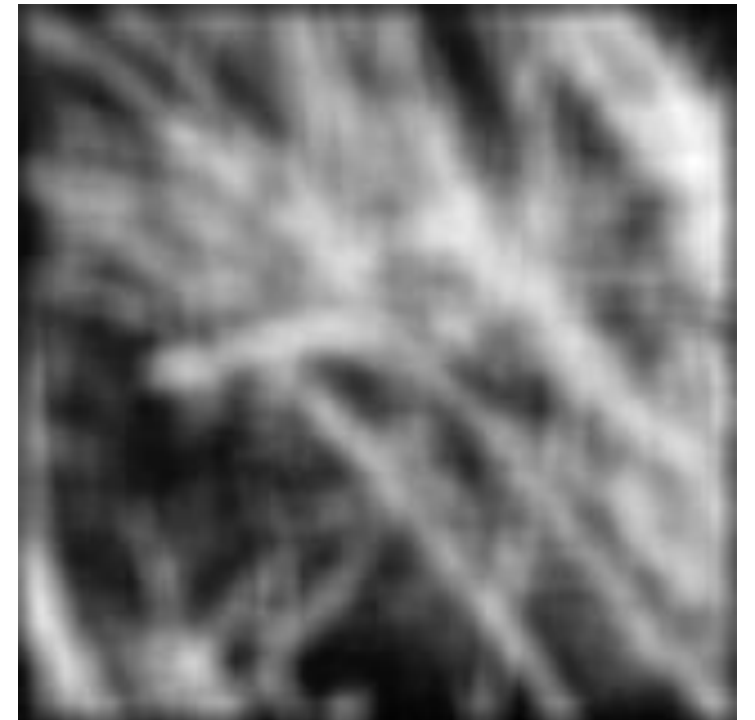
depicts filter kernels:
white = high value
black = low value



original



filtered
(Gaussian kernel)



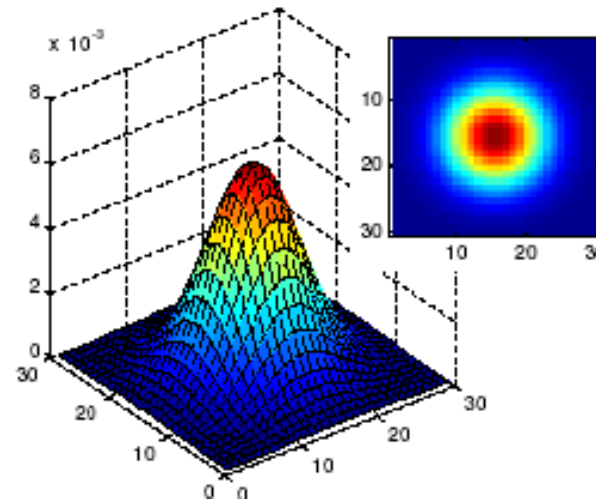
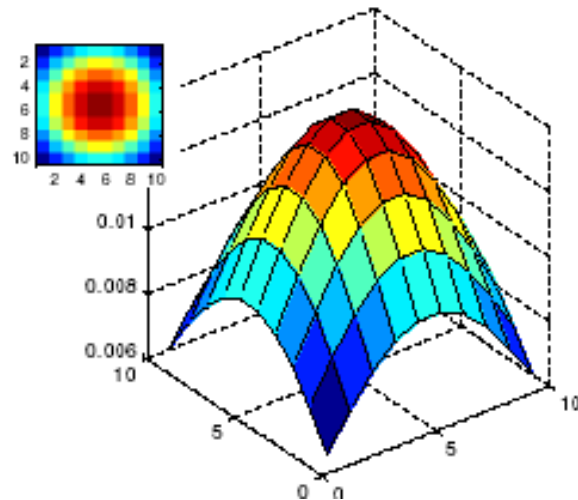
filtered
(Box kernel)

Gaussian filters

Which parameters matter?

Size of kernel?

Note, Gaussian function has infinite support, but discrete filters use finite kernels

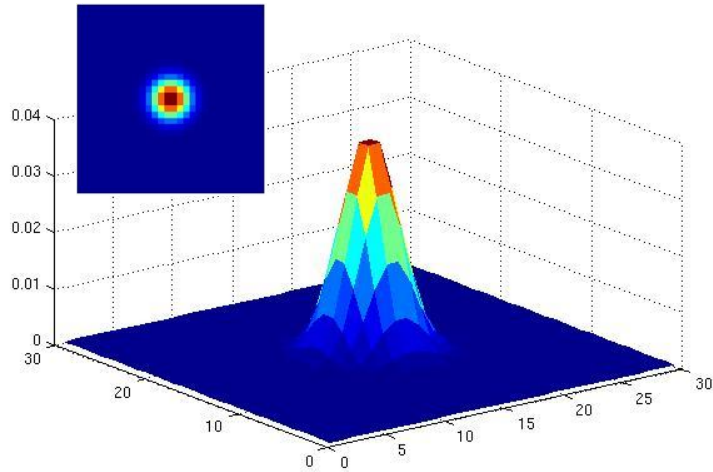


$\sigma = 5$ with 10 x 10 kernel $\sigma = 5$ with 30 x 30 kernel

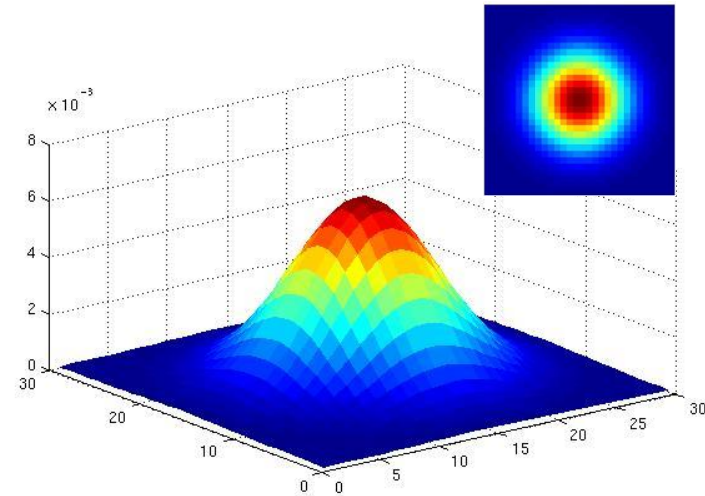
Gaussian filters

Which parameters matter?

Variance of Gaussian (σ): determines extent of smoothing



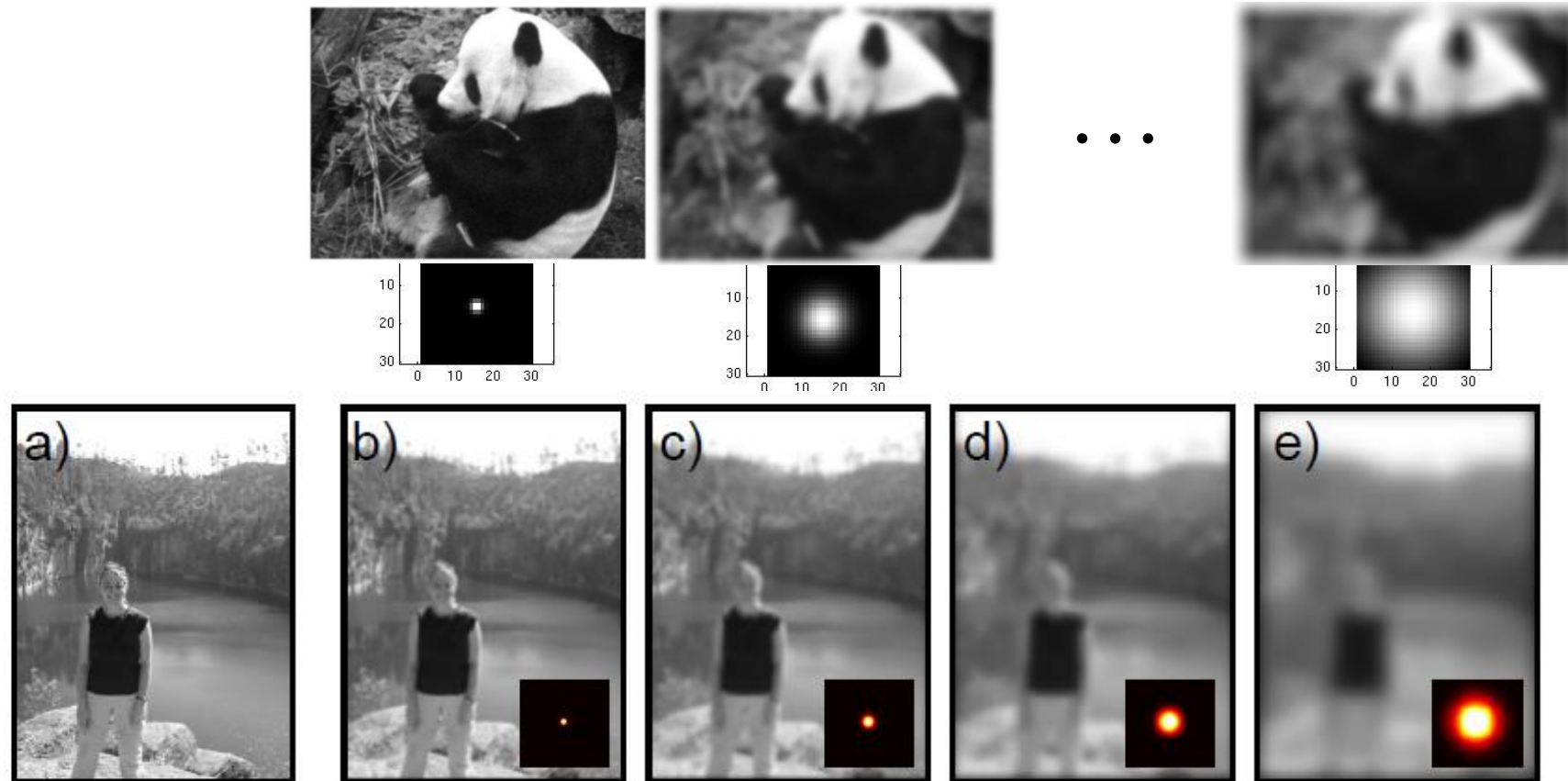
$\sigma = 2$ with 30×30 kernel



$\sigma = 5$ with 30×30 kernel

Smoothing with a Gaussian

σ is the “scale” of the Gaussian kernel, and controls the amount of smoothing.



Practice with Correlation Filters



Original



0	0	0
0	1	0
0	0	0



Filtered (no change)



Original



0	0	0
0	0	1
0	0	0



Filtered (shifts to the left)



Practice with Correlation Filters: Sharpening


$$\otimes \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} -1 & -1 & -1 \\ -1 & 17 & -1 \\ -1 & -1 & -1 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \right) =$$


Original



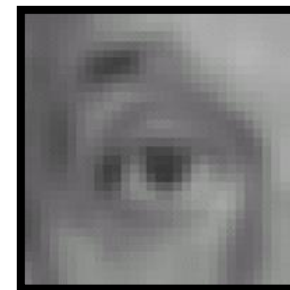
Original

+



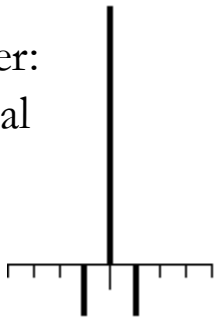
Original

-



Blurred (box filter)

Sharpening filter:
accentuates local
differences



“Details” = Original - Blurred

Sharpening Emphasizes Local Differences

input



filter

$$- \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -17 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

output



sharpening

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

Response if region is “flat”?

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

Region has an edge?

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

Region has noise?

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

No change

0	-2	14	12	14
0	-3	12	9	12
0	-3	12	9	12
0	-3	12	9	12
0	-2	14	12	14

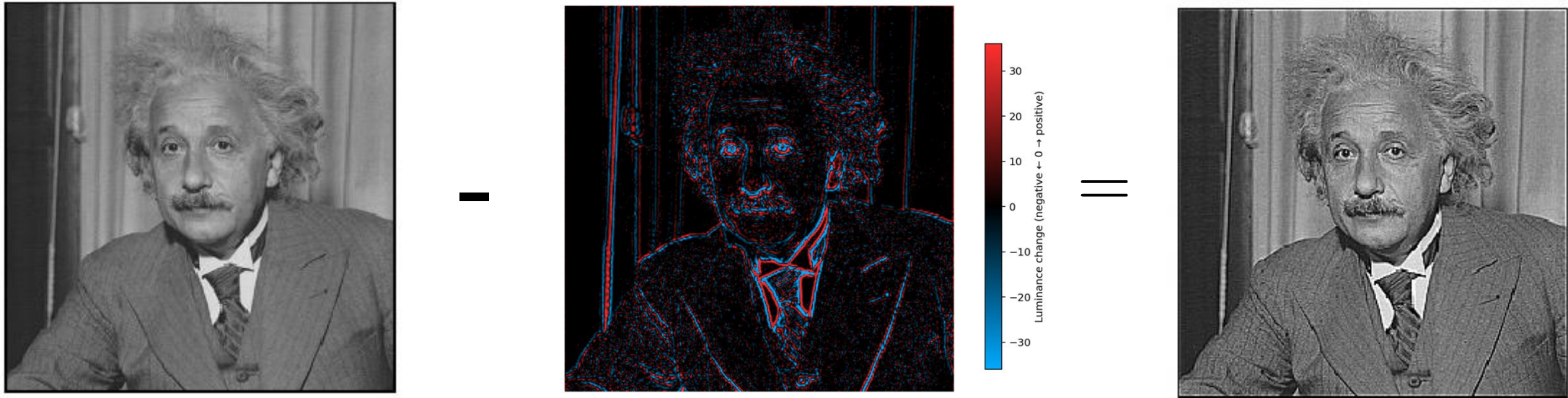
Edge gets emphasized

02. Point Processing & Filtering

0	0	0	0	0
0	-1	-1	-1	0
0	-1	17	-1	0
0	-1	-1	-1	0
0	0	0	0	0

Noise also emphasized ☹️

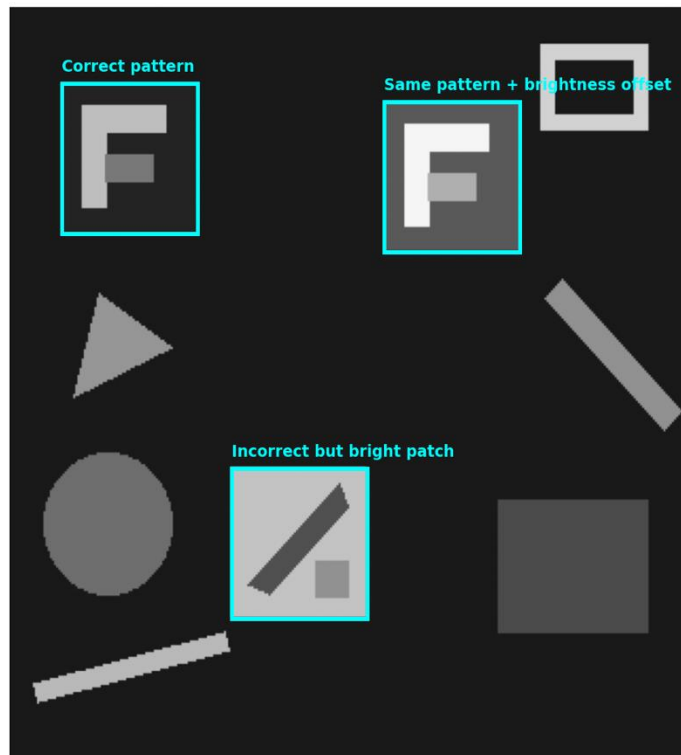
Emphasizing Local Differences



- It does nothing for flat areas
- It stresses intensity peaks and differences with respect to the surroundings

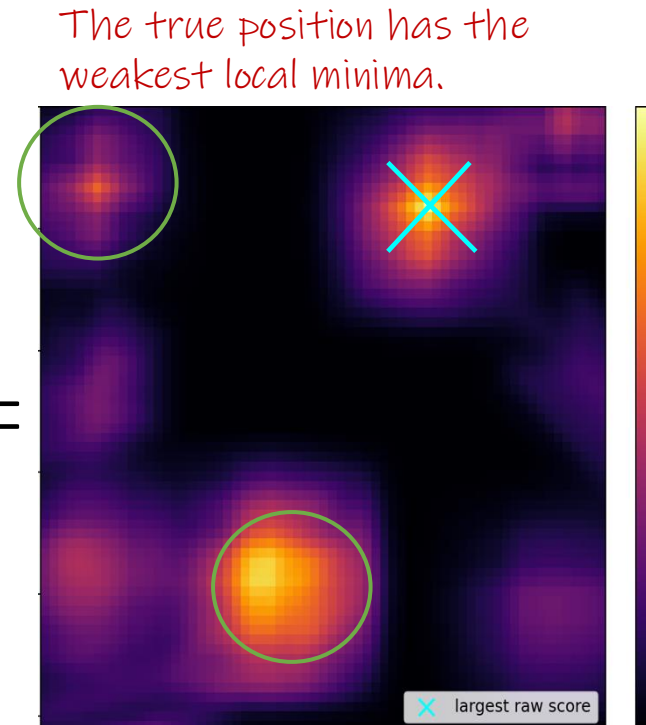
Filters as Local Pattern Detectors

Find  in the image with Cross-Correlation:



P

$$\otimes \begin{matrix} \text{F} \\ F \end{matrix} =$$



The true position has the weakest local minima.

The largest response comes from an increased brightness applied to the matching pattern.

Naïve cross-correlation leads to generally bright regions which dominate even the true match.

Normalized Cross-Correlation

- The brightness is misleading?
Remove the averages $\bar{\mathbf{w}}_{ij}$, $\bar{\mathbf{F}}$

$$\mathbf{w}_{ij} - \bar{\mathbf{w}}_{ij}$$

Patch / window within input image,
centered at $[i,j]$ adjusted by average

$$\mathbf{F} - \bar{\mathbf{F}}$$

Average-adjusted kernel

- Normalize by the l_2 norm $\|\cdot\|$

$$\hat{\mathbf{w}}_{ij} = \frac{\mathbf{w}_{ij} - \bar{\mathbf{w}}_{ij}}{\|\mathbf{w}_{ij} - \bar{\mathbf{w}}_{ij}\|}$$

$$\hat{\mathbf{F}} = \frac{\mathbf{F} - \bar{\mathbf{F}}}{\|\mathbf{F} - \bar{\mathbf{F}}\|}$$

l_2 norms: sqrt of the sum of the squared elements

- Compute the weighted sum

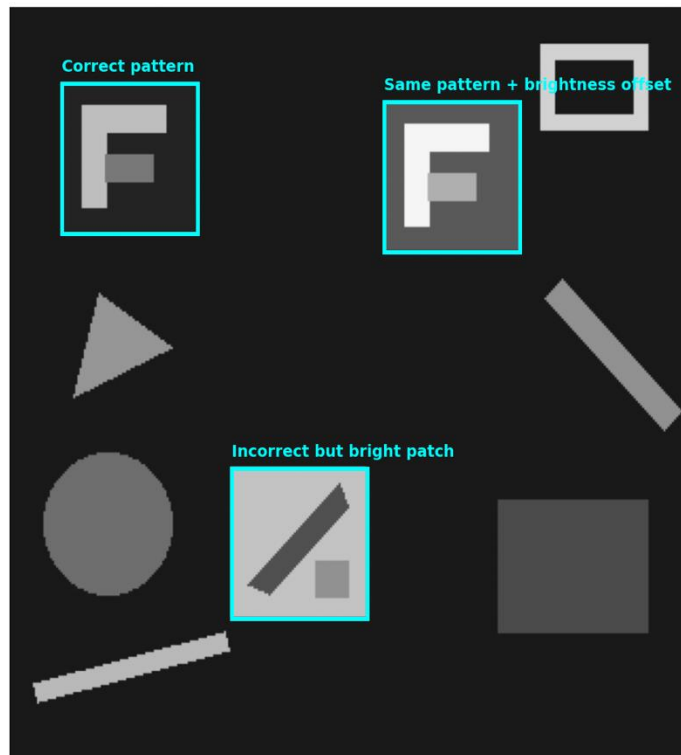
$$x_{ij} = \hat{\mathbf{w}}_{ij}^T \hat{\mathbf{F}} = \sum_{u,v} \hat{\mathbf{w}}_{ij} \hat{\mathbf{F}}$$

Sum the element-wise multiplication;
note the matrix notation alternative.



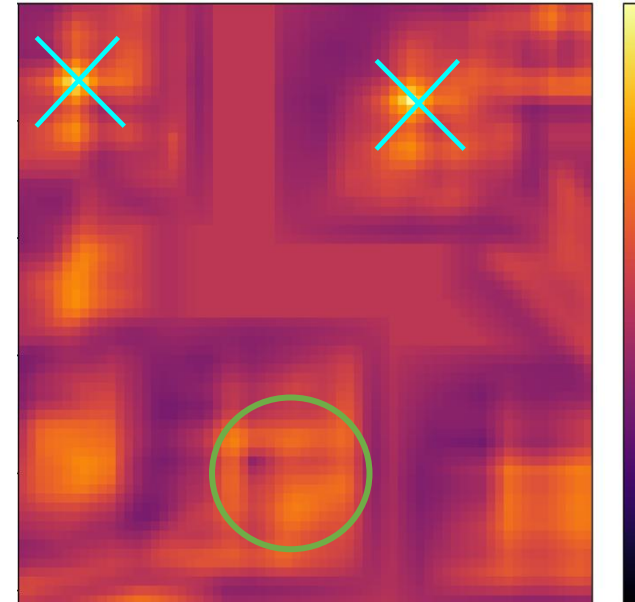
Fixed Local Pattern Detector

Find  in the image using **Normalized-Cross-Correlation**:



$$\otimes \text{F} =$$

The true position has one strong local minima.



Brightness offset is not distinguishable and has an equally strong response as the correct pattern.

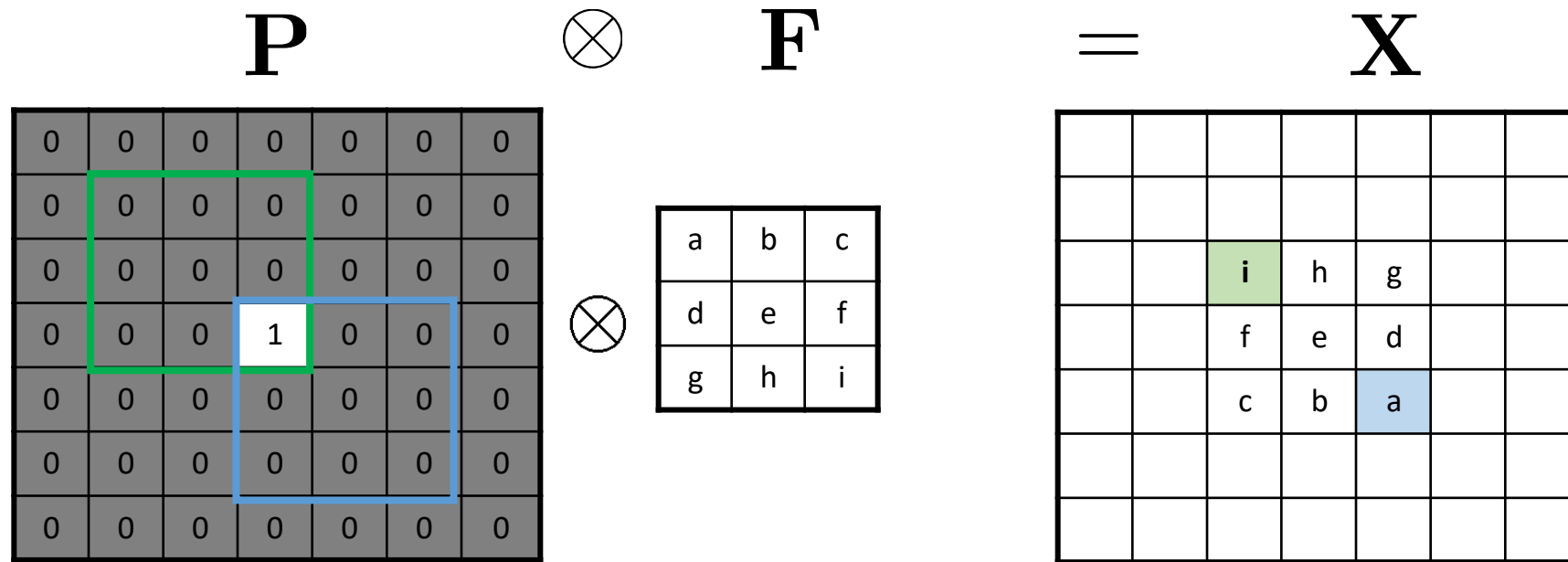
Normalized cross-correlation removes the impact of generally bright regions.

Bridge to Modern CNNs

Convolution operation; convolution vs. cross-correlation;
kernels, stride, padding and channels.

Filtering an impulse signal

What is the result of filtering an impulse signal (image) with an arbitrary kernel?



Convolution

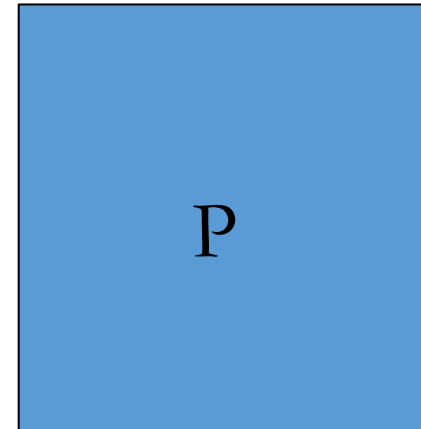
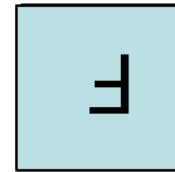
Flip the kernel in both dimensions, then apply cross-correlation

$$x_{ij} = \sum_{u=-k}^k \sum_{v=-k}^k f_{uv} \cdot p_{i-u, j-v}$$

Subtraction of u, v indices!

$$\mathbf{X} = \mathbf{F} * \mathbf{P}$$

convolution operator





Convolution vs. Cross-Correlation (1)

Note the strong similarities in the two operations.

Convolution

$$x_{ij} = \sum_{u=-k}^k \sum_{v=-k}^k f_{uv} \cdot p_{i-u, j-v} \quad \mathbf{X} = \mathbf{P} * \mathbf{F}$$

Cross-correlation

$$x_{ij} = \sum_{u=-k}^k \sum_{v=-k}^k f_{uv} \cdot p_{i+u, j+v} \quad \mathbf{X} = \mathbf{P} \otimes \mathbf{F}$$

Convolution vs. Cross-Correlation (2)

Cross-correlation

A form of **recognition**

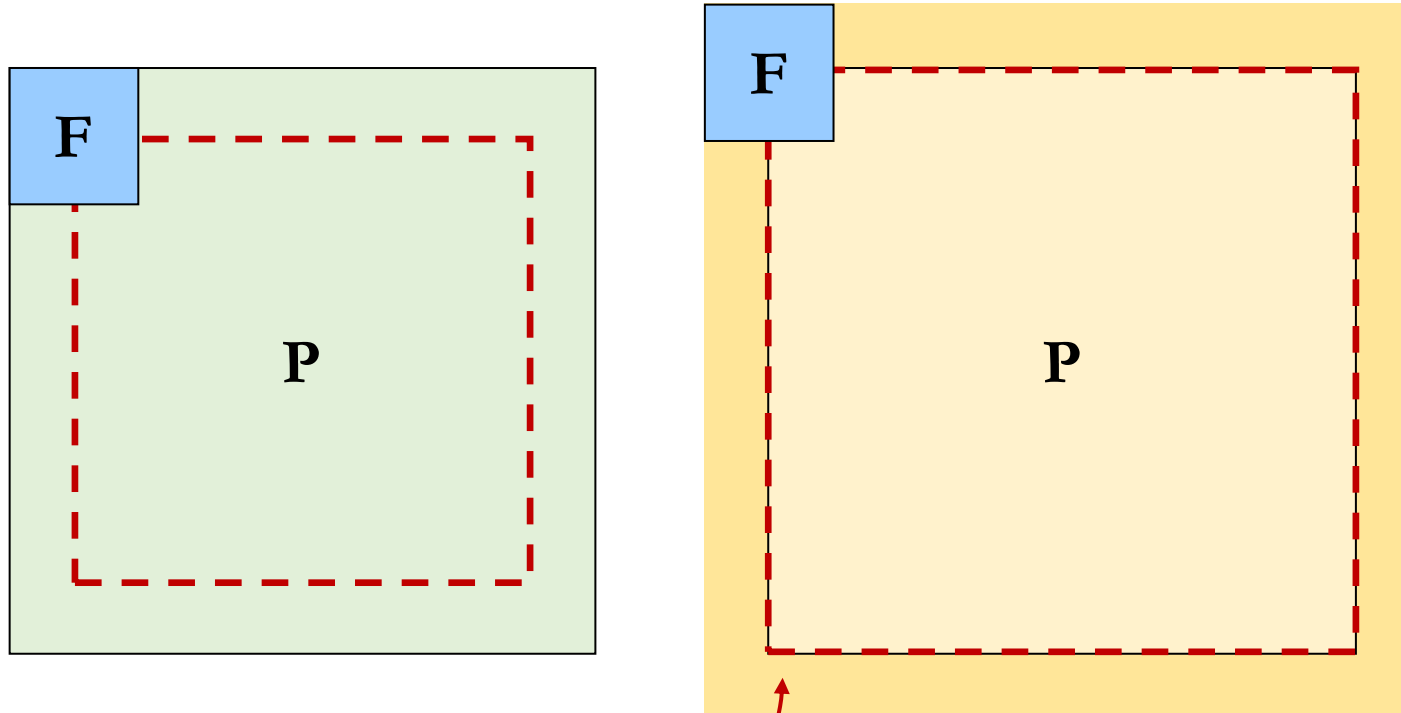
- Measuring similarity to a reference pattern
- *“Does this region look like my template here?”*
- Equivalent to convolution operation if kernel is symmetric

Convolution

An **image transformation**

- Applies a local transform (filter) everywhere
- *“How does each pixel affect (and get affected by) its neighbours?”*
- Special mathematical properties which supports filter composition and an equivalent frequency domain representation

Output Size & Boundaries



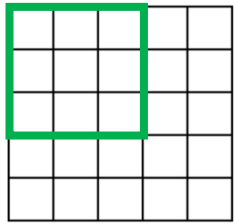
Filter window falls off
the edge of the image
– what goes in here?



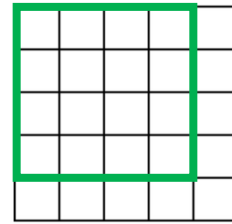
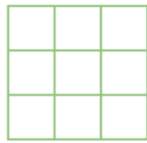
Padding extends the image border.
0-padding is the most common.

2D Convolution (Hyper-) Parameters

Kernel size: 3x3, 5x5



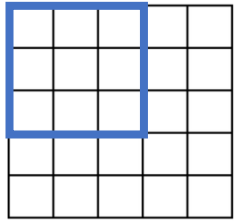
=



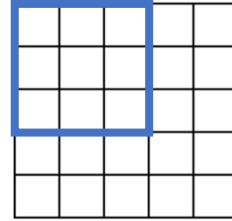
=



Stride: How far the kernel moves per step.



=

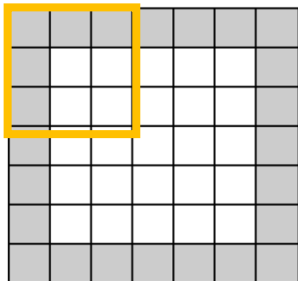


=

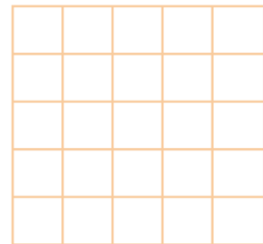


A stride of 1 is the default per-pixel sweep; a stride of 2 means the kernel "jumps" 1 pixel or element.

Padding: Extent (how much padding) and type (zero, copy, etc.)



=



2D Convolution (Hyper-) Parameters

n_{in}/n_{out} - input / output sizes

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

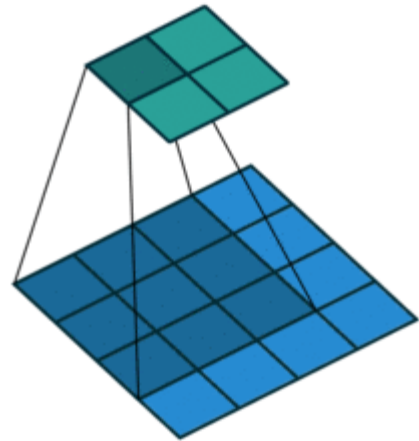
“Valid”: no padding.
Applies convolution
wherever it is valid.

“Same” : pads to ensure
that output size will
match the input size.

“Full”: pads to ensure that every
input element is used for all
possible convolution locations.

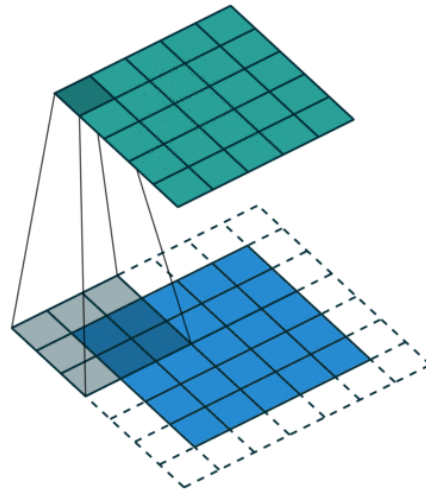
Y

X



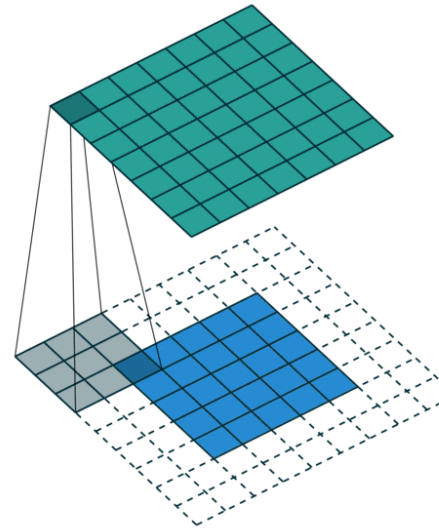
$k=3, p=0, s=1$ (Valid)

Kernel = 3x3
Padding = 0
Stride = 1



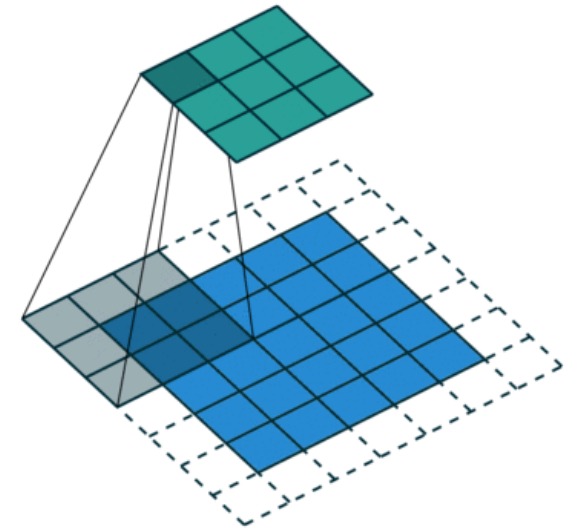
$k=3, p=1, s=1$ (Same)

Kernel = 3x3
Padding = 1
Stride = 1



$k=3, p=2, s=1$ (Full)

Kernel = 3x3
Padding = 2
Stride = 1

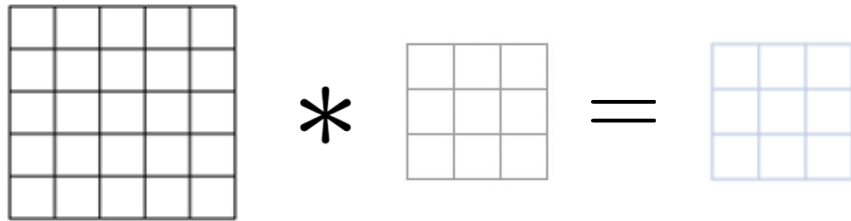


$k=3, p=1, s=2$

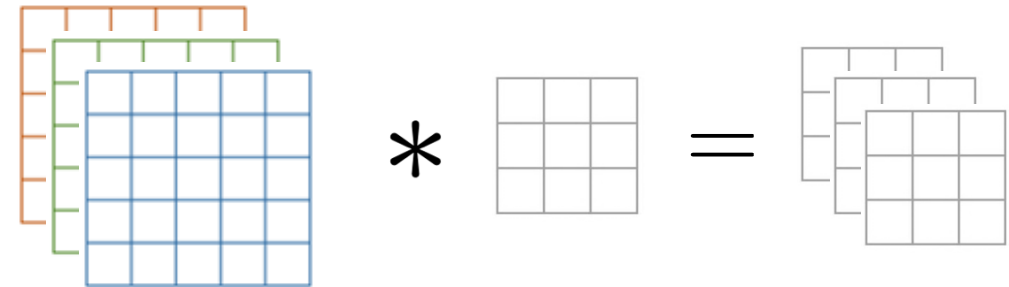
Kernel = 3x3
Padding = 1
Stride = 2

Channel Depth

- The # of channels in an image (or a kernel) is sometimes referred to as depth, or channel depth
 - Greyscale images have channel depth of 1
 - RGB images, with 3 channels, have a channel depth of 3



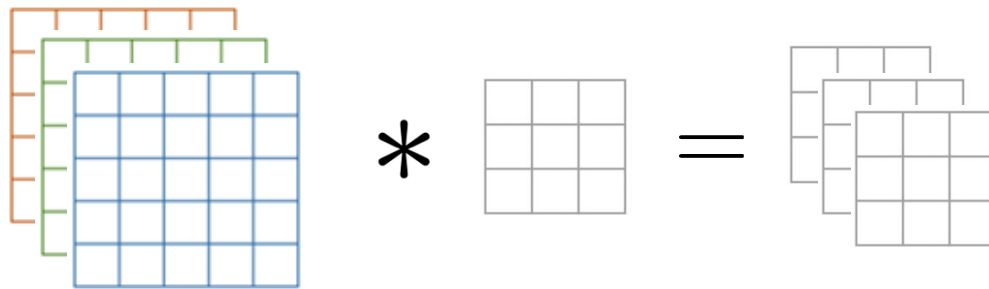
1-channel input convolved with 1-channel kernel leads to a 1-channel output



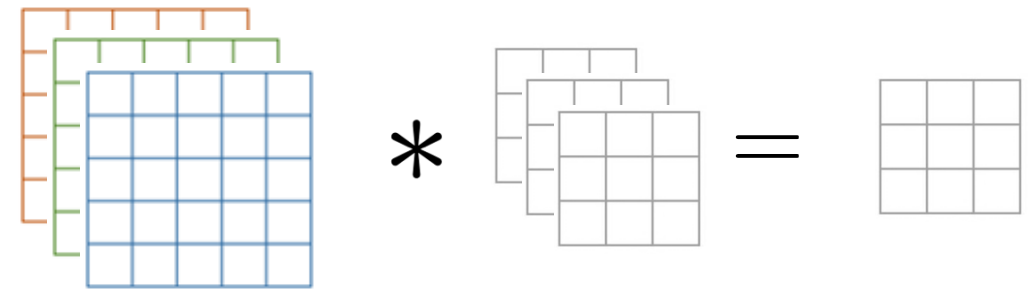
3-channel input convolved with 1-channel kernel leads to a 3-channel output

Kernel Depth

- The kernel's dimensionality specifies the neighbourhood extent from which we aggregate information.
 - Single-channel kernels gather (spatial) information from a single channel
 - Multi-channel kernels gather (spatial) information across the channels



1-channel kernel

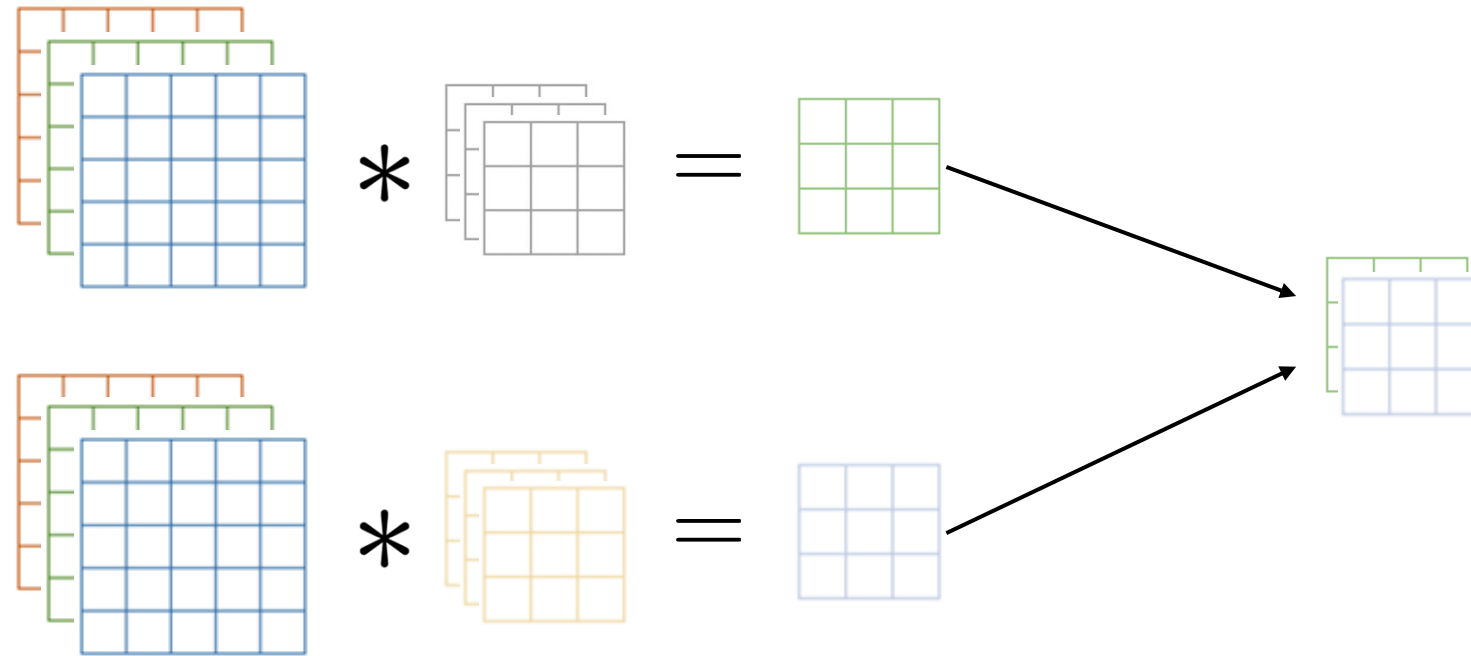


3-channel kernel

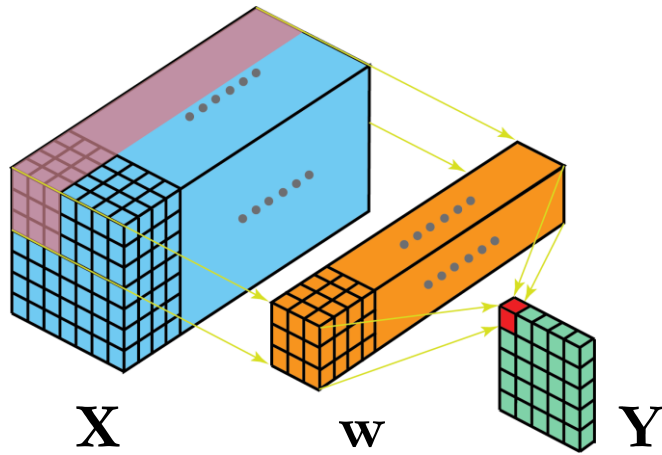
Most vision applications with colour images or multi-channel inputs match the # number of channels in the kernel to the # of channels in the input.

Multiple Kernels

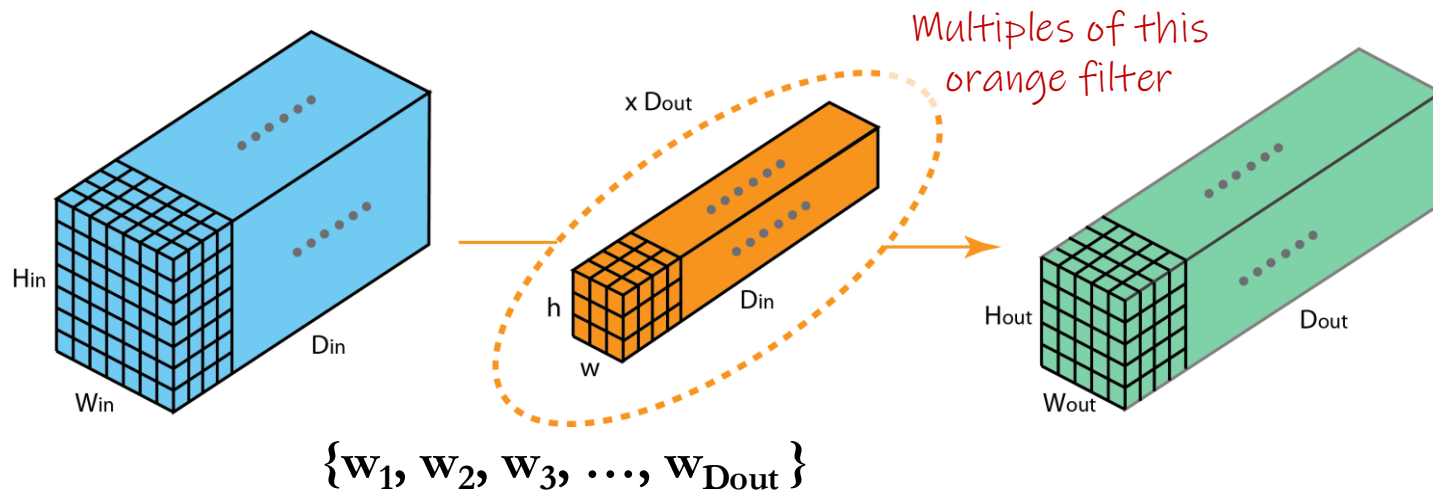
Grouping multiple (multi-channel) kernels leads to multi-channel outputs. This forms the basis of a filter bank (more on this in Lecture 4).



2D Convolution on Multi-Channel Data



- 2D convolution slides a multi-channel kernel or filter through the input layer.
- The input layer and filter have the same depth or number of channels.
- Multi-channel filter moves only in 2 directions: height & width of the input (hence being a 2D convolution).
- Output here is a one-layer matrix.”



- For multi-channel output, apply multiple 3D filter kernels

In-Lecture Question:



Non-Linear Filters

Median filtering, denoising impulse noise, max filtering

Non-Linear Filters

- Correlation and convolution are linear operations
- Non-linear filters perform non-linear operations, e.g.
 - Median
 - Median filtering is commonly used for removing impulse and salt & pepper noise
 - Min/Max
 - The max operation is used in max-pooling for deep neural networks.
 - [Morphology](#)

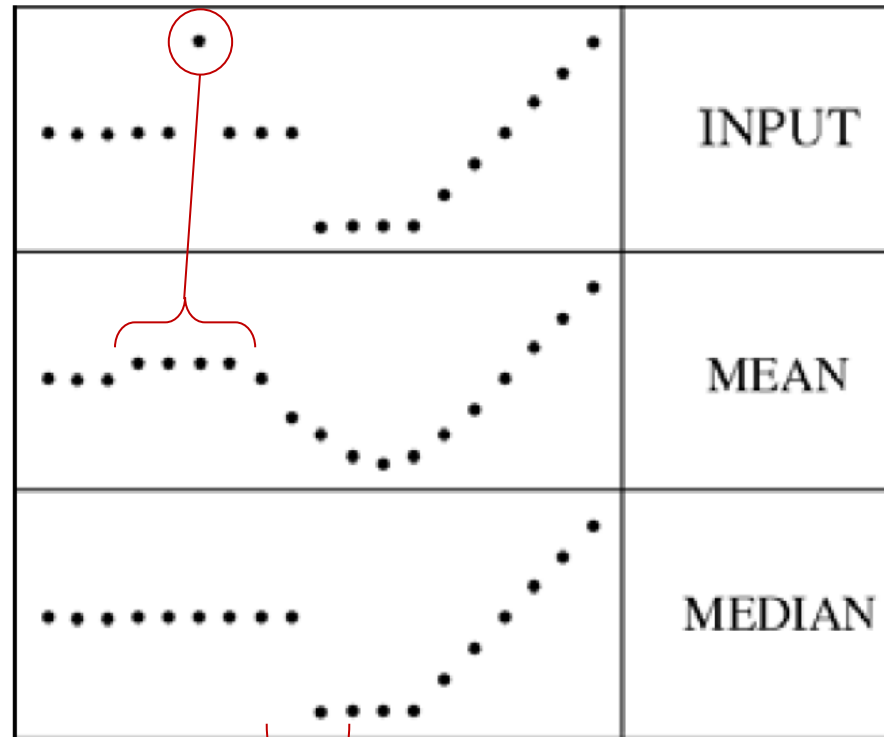
Denoising: Median Filter



Salt and pepper noise

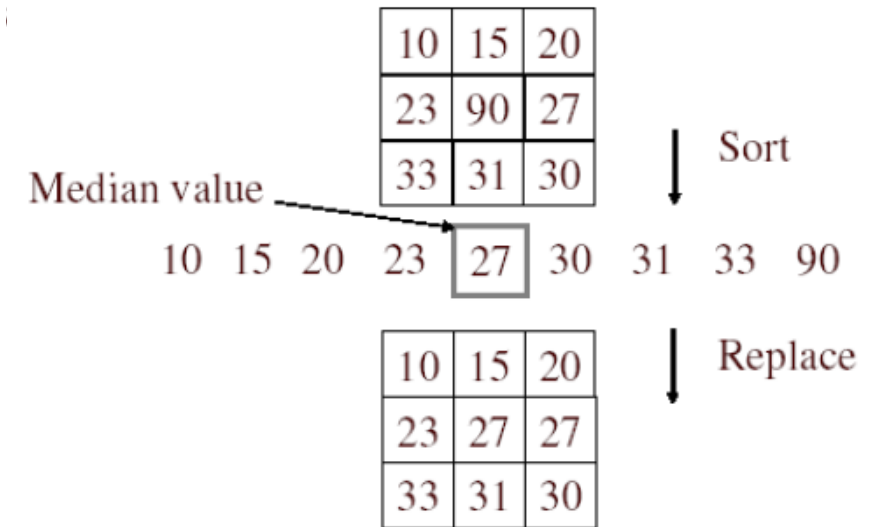


Impulse noise

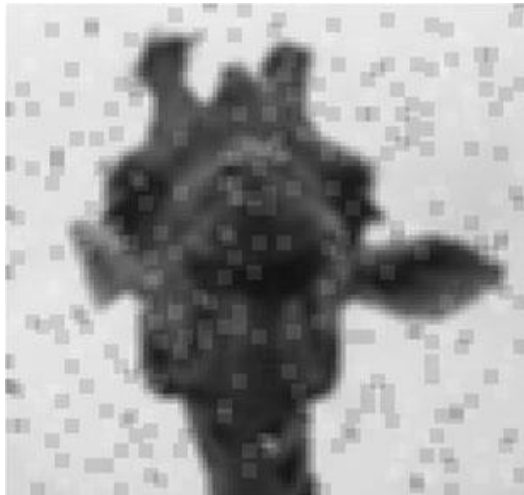
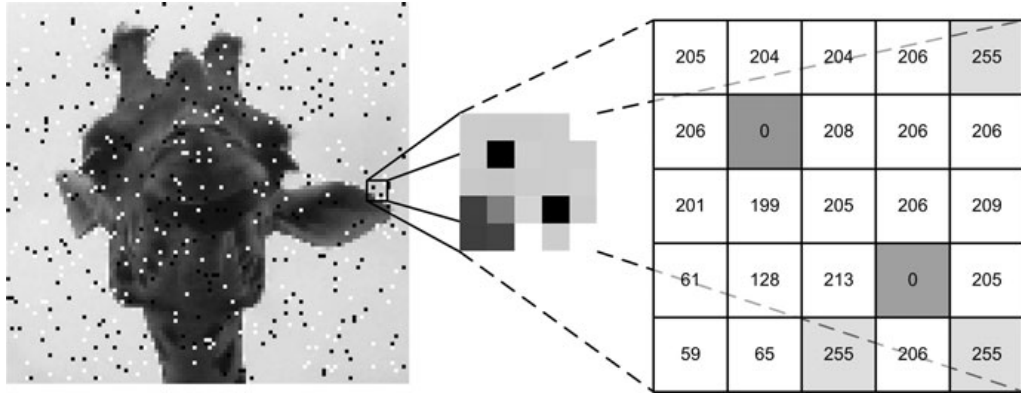


edge-
preserving

- Applies median operation to a local neighbourhood
- No new pixel values introduced
- Removes spikes: good for impulse, salt & pepper noise



Median Filter Outputs

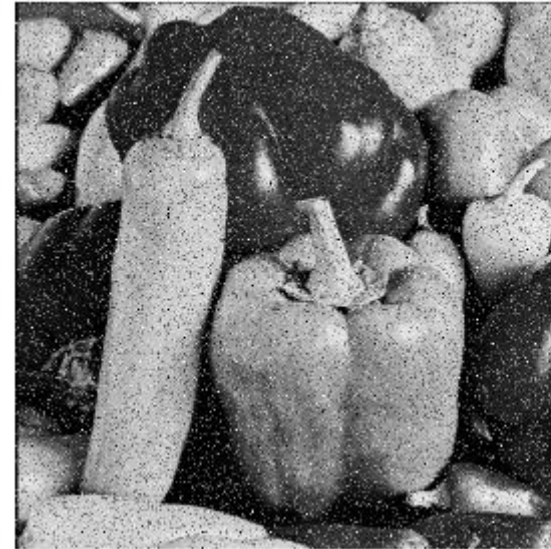


Mean filtered

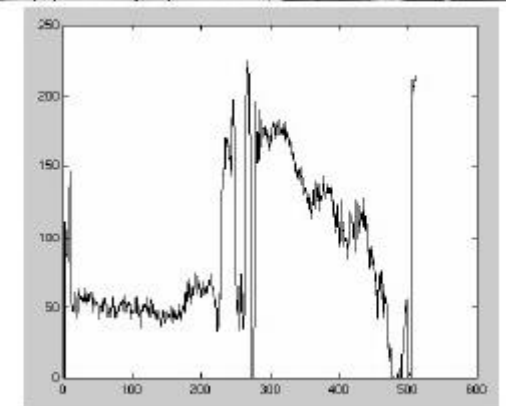
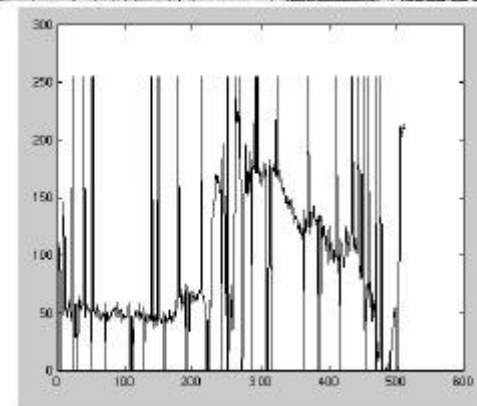


Median filtered

Salt and pepper noise

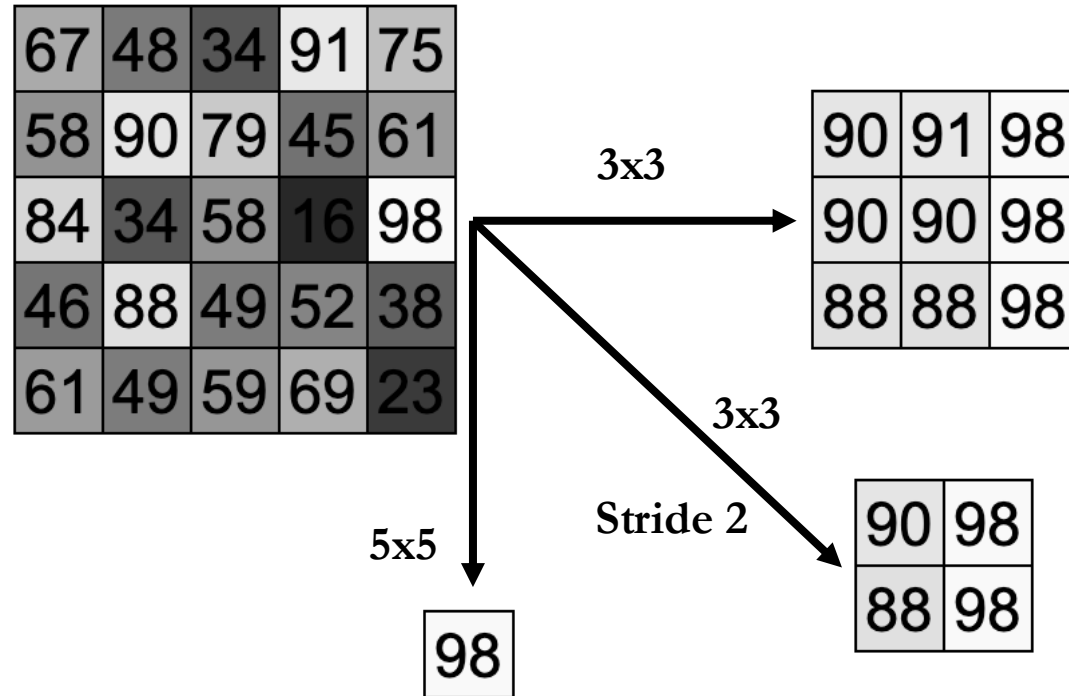


Median filtered

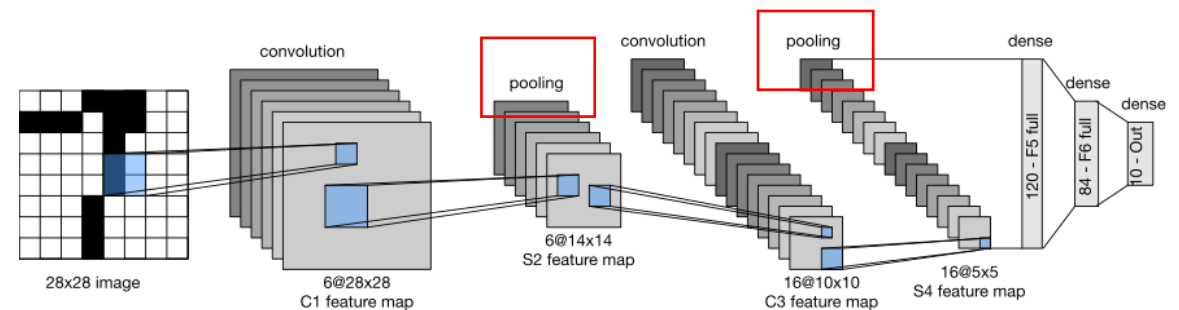


Plots of a scanline.

Max filtering



- Applies the max operation to a local neighbourhood
- Max filtering, with appropriate stride, forms the basis of the max-pooling operation used in CNNs
- Max pooling “**summarizes**” a neighborhood in a feature map





Max filtering

The maximum filter lets the brightest pixel in a neighborhood "take over" its neighbors.



3x3



5x5



7x7



9x9

Summary

1. Point-wise Processing

- Brightness, contrast, normalization, non-linear mappings
- Histograms: stretching, equalization, segmentation

2. Linear Filtering

- Achieves many effects: denoising, sharpening, moving, template matching
- (Cross-)Correlation:
 - weighted summation of input window, weights stored in filter / kernel
 - normalized version for template matching
- Convolution
 - Flips kernel before applying cross-correlation
 - Kernel usage and parameters

3. Non-linear filters

- Median filter for smoothing salt & pepper, impulse noise
- Max filter for patch summarization

Extra Reading

- [Moeslund](#) Chapter 4 Point Processing
- [Moeslund](#) Chapter 5 Neighbourhood Processing

- [Szeliski](#) Chapter 3 Image Processing

- [Torralba](#) Chapter 15 Linear Image Filtering
- [Torralba](#) Chapter 17 Blur Filters

AI-Generated FAQs

FAQ 1: Intensity Transformations

Q: How does histogram stretching differ from image standardization?

A: Stretching maps selected low and high intensities to a display range such as 0 to 255.

Standardization changes the values to have a chosen mean and standard deviation, commonly zero and one. Standardized values are not necessarily display-ready.

Simple standardization is not necessarily whitening. For multichannel data, whitening also removes correlations between channels or features.

Q: Can histogram equalization reveal information that was not captured in the original image?

A: No. It can make recorded intensity differences easier to see, but it cannot recover information lost through clipping, saturation, or coarse quantization. It may also make noise and compression artifacts more visible.

FAQ 2: Smoothing and Sharpening

Q: If sigma controls the width of a Gaussian, why must we also choose the kernel size?

A: An ideal Gaussian extends infinitely, but a computer must truncate it to a finite window. The kernel must be large enough to contain the significant weights associated with the chosen sigma. Once the omitted weights are negligibly small, making the kernel larger has little practical effect. A common choice is a kernel width of approximately six times sigma, rounded up to the next odd integer.

Q: If the noise is Gaussian, does that mean a Gaussian filter removes it?

A: No. Gaussian noise describes the distribution of the noise values. A Gaussian filter describes the spatial weights used to average neighboring pixels.

Smoothing reduces noise variation, but it also removes genuine image detail. If the noise level is unknown, there is no uniquely correct sigma: more smoothing suppresses more variation but introduces more blur.

Q: Sharpening involves subtraction, so why do we say that detail is added back?

A: Subtraction first extracts the detail: $\text{detail} = \text{original} - \text{blurred}$

The detail is then added to the original: $\text{sharpened} = \text{original} + \alpha \times \text{detail}$

Alpha controls the strength of the sharpening.

FAQ 3: Pattern Matching and Convolution

Q: Why should a template match produce a local maximum, and why is normalization necessary?

A: After the template and image window have been centred and normalized, their correlation score measures pattern agreement rather than overall brightness. Identical normalized patterns give the largest possible score. Without normalization, bright or high-energy image regions may produce large raw correlation values even when their patterns do not match the template well.

Q: Is normalized cross-correlation the same as making the kernel weights sum to one?

A: No. Making smoothing-kernel weights sum to one preserves constant image regions.

Normalized cross-correlation instead subtracts the mean and normalizes the magnitude of both the template and each local image window.

Q: Why does convolution flip the kernel? Does flipping improve template matching?

A: Flipping is part of the mathematical definition of convolution; it is not intended to improve template matching. Template matching is normally performed using cross-correlation, which does not flip the template. Convolution is useful because it has convenient properties for representing and combining linear, shift-invariant operations. For symmetric kernels, convolution and cross-correlation produce the same result.

Q: Can the image and filter be swapped?

A: Mathematical convolution is commutative, so swapping the two operands gives the same full convolution. Cross-correlation is not generally commutative.

In image-processing software, the image and kernel still have different practical roles, and output size and boundary handling are defined relative to those roles.

AI-Generated FAQs

FAQ 4: Output Size and CNN Operations

Q: How much padding is needed? Why do different sources give different answers?

A: The answer depends on the desired output size. For input size N_{in} , kernel size K , padding p on each side, and stride 1:

$$N_{out} = N_{in} + 2p - K + 1$$

Valid:

$$p = 0$$

$$N_{out} = N_{in} - K + 1$$

Same:

$$p = (K - 1)/2$$

$$N_{out} = N_{in}$$

Full:

$$p = K - 1$$

$$N_{out} = N_{in} + K - 1$$

These same-padding expressions assume an odd kernel size.

Q: If a 3-by-1 kernel convolved with a 1-by-3 kernel produces a 3-by-3 kernel, why can convolution still produce a same-sized image?

A: These statements concern different outputs. Combining the two kernels using full convolution produces a 3-by-3 composite kernel.

When that composite kernel is subsequently applied to an image, padding and cropping determine whether the image output is valid, same, or full size.

Q: When a filter is applied to an RGB image, does each input channel produce a separate output channel?

A: A conventional filter has one K -by- K slice for each input channel. The responses from all input channels are summed, so one filter produces one output channel.

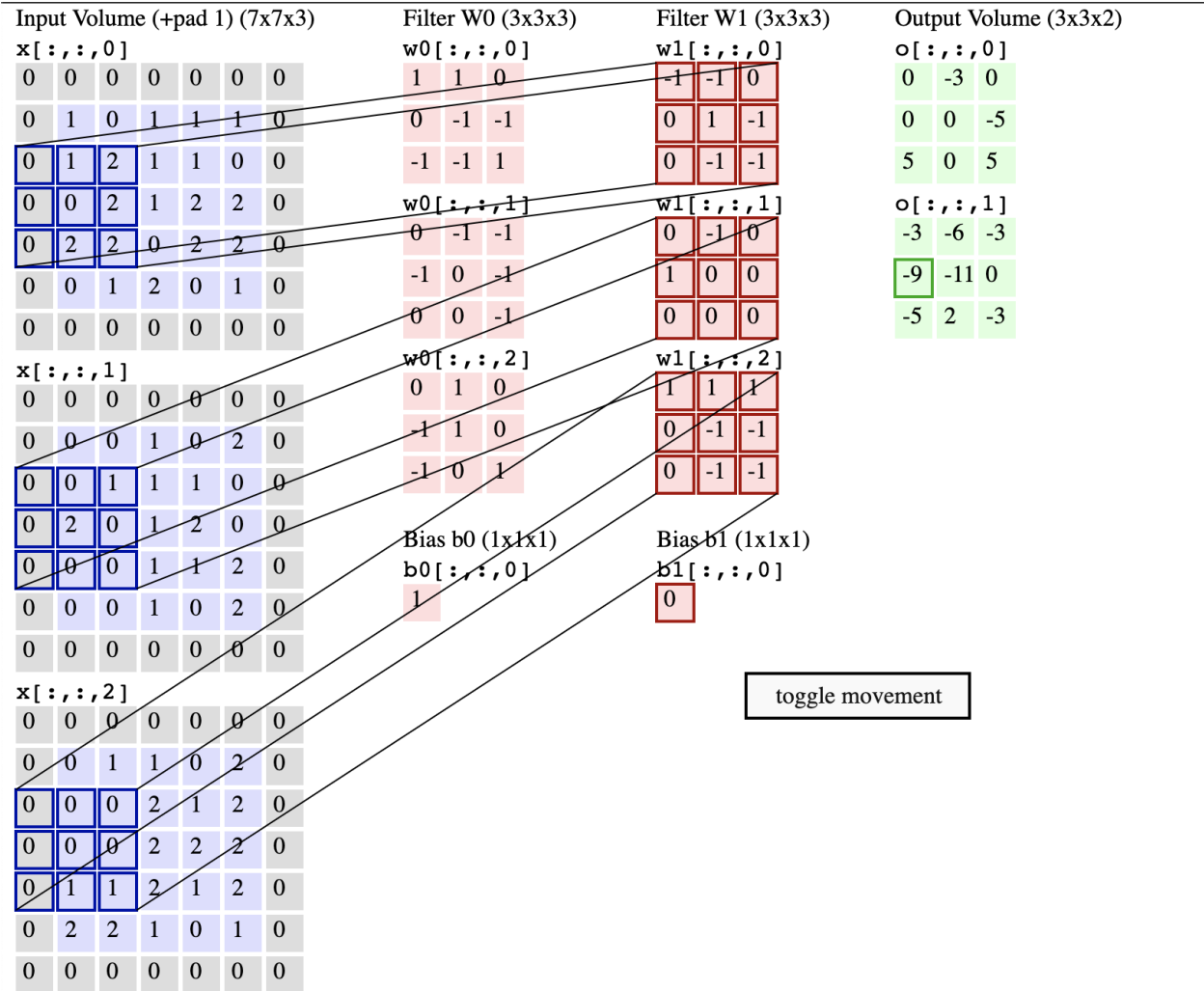
Applying multiple filters produces multiple output channels.

Q: Does max filtering always reduce the spatial dimensions?

A: No. A stride-1 max filter can preserve the input dimensions by using suitable padding.

Spatial reduction occurs when the maximum is computed with a larger stride or followed by subsampling. This operation is commonly called max pooling.

2D Convolution (w/ 3D kernels and data)



Demo [\[link\]](#)
Visualization [\[link\]](#)